

# APRENDIZAJE DE MODELOS DE ORDEN REDUCIDO ROBUSTOS MEDIANTE EL MÉTODO ADJUNTO

FRANCISCO GARCÍA ATIENZA

Trabajo de Fin de Máster  
2025:E20



LUND UNIVERSITY

Facultad de Ciencias  
Centro de Ciencias Matemáticas  
Análisis Numérico



# APRENDIZAJE DE MODELOS DE ORDEN REDUCIDO ROBUSTOS MEDIANTE EL MÉTODO ADJUNTO

FRANCISCO GARCÍA ATIENZA

Trabajo de Fin de Máster  
2025:E20



LUND UNIVERSITY

Facultad de Ciencias  
Centro de Ciencias Matemáticas  
Análisis Numérico



# AGRADECIMIENTOS

Me gustaría expresar mi gratitud a mis compañeros de clase y profesores durante estos últimos cinco años de estudios de grado y máster, sin ningún orden en particular: Claus, Philipp, Robert, Tony, Erik, Anna-Maria, Viktor, Kjell, Anitha, Jan-Fredrik y muchos otros. De cada uno de vosotros he aprendido algo único, y todos habéis contribuido a profundizar mi pasión por las matemáticas.

Un agradecimiento especial a mi supervisor, Mengwu Guo, por su cercanía, su inestimable orientación y por introducirme en el fascinante mundo del Aprendizaje Automático Científico (Scientific Machine Learning).

Por último, aunque no por ello menos importante, dedico esta tesis a mi familia. A mis seis hermanos, a quienes quiero con todo mi corazón; a mi padre, siempre presente en mi memoria, quien me enseñó el valor de la humildad en la educación; a mi madre, por mostrarme cada día lo que significa sacrificarse por aquello que más se ama; y a mi compañero de vida durante este camino, Alfredo, por su amor incondicional tanto en los buenos como en los malos momentos. Vuestro apoyo ha sido mi mayor fortaleza.

*Lund, Junio de 2025*

*Francisco García Atienza*



# RESUMEN

Los modelos de orden reducido (Reduced-Order Models, ROMs) se han convertido en herramientas indispensables para disminuir la complejidad computacional de simulaciones de alta fidelidad en ciencia e ingeniería. En esta tesis, presentamos un novedoso marco de entrenamiento que combina la formulación integral de la Inferencia de Operadores (Operator Inference, OpInf) con métodos de estado adjunto (adjoint-state methods) para obtener ROMs robustos y basados en datos. Al formular un funcional de pérdida en tiempo continuo que integra las ecuaciones gobernantes a lo largo del tiempo, nuestro enfoque evita la diferenciación explícita de datos de estado ruidosos y regulariza de forma inherente el problema de aprendizaje. Derivamos las ecuaciones adjuntas correspondientes para el cálculo exacto del gradiente del funcional de pérdida y presentamos un algoritmo de descenso por gradiente que actualiza eficientemente los parámetros del ROM.

Validamos el método propuesto en dos ecuaciones en derivadas parciales (EDP) no lineales canónicas: la ecuación de Burgers viscosa y la ecuación de reacción-difusión de Fisher-KPP bidimensional. Mediante experimentos numéricos sistemáticos, comparamos el ROM entrenado mediante el método adjunto con el enfoque estándar de OpInf bajo dos escenarios: (i) diferentes densidades de instantáneas temporales (snapshots) y (ii) la presencia de ruido gaussiano aditivo. Nuestros resultados muestran que ambos métodos presentan un rendimiento comparable cuando las instantáneas se submuestran uniformemente; sin embargo, el método adjunto exhibe una mayor precisión y estabilidad en presencia de datos ruidosos. Además, aunque cada iteración de entrenamiento requiere una pasada hacia atrás (backward pass), el coste computacional total escala de forma constante con respecto al número de parámetros, lo que convierte al método en una alternativa especialmente atractiva para modelos completos de muy alta dimensión.



# ABSTRACT

Reduced-order models (ROMs) have become indispensable tools for reducing the computational complexity of high-fidelity simulations in science and engineering. In this thesis, we introduce a novel training framework that combines the integral form of Operator Inference (OpInf) with adjoint-state methods to yield robust, data-driven ROMs. By formulating a continuous-time loss functional that integrates the governing equations over time, our approach avoids explicit differentiation of noisy state data and inherently regularizes the learning problem. We derive the corresponding adjoint equations for exact gradient computation of the loss functional and present a gradient-descent minimization algorithm that updates ROM parameters efficiently.

We validate the proposed method on two canonical nonlinear PDEs: the viscous Burgers' equation and the two-dimensional Fisher-KPP reaction-diffusion equation. In systematic numerical experiments, we compare the adjoint-trained ROM against the standard OpInf approach under (i) varying snapshot density and (ii) additive Gaussian noise. Our results show that both methods perform comparably when snapshots are uniformly sub-sampled, but the adjoint method exhibits improved accuracy and stability in the presence of noisy data. Moreover, although each training iteration requires a backward pass, the overall computational cost scales constantly with the number of parameters, making the method favorable for very high-dimensional full models.



# ÍNDICE GENERAL

|                                                                         |           |
|-------------------------------------------------------------------------|-----------|
| <i>Índice de figuras</i>                                                | IX        |
| <b>1. Introducción</b>                                                  | <b>1</b>  |
| <b>2. Preliminares</b>                                                  | <b>5</b>  |
| 2.1. Reducción de Orden de Modelos (MOR) . . . . .                      | 5         |
| 2.2. Inferencia de Operadores (OpInf) . . . . .                         | 8         |
| <b>3. Método Adjunto para la Inferencia de Operadores</b>               | <b>13</b> |
| 3.1. Un Funcional de Pérdida en Tiempo Continuo . . . . .               | 13        |
| 3.2. Las Ecuaciones del Estado Adjunto . . . . .                        | 15        |
| 3.3. Ecuaciones de los Tres Gradientes . . . . .                        | 24        |
| 3.3.1. $\nabla_{\hat{\mathbf{q}}} \hat{\mathbf{f}}$ . . . . .           | 24        |
| 3.3.2. $\nabla_{\hat{\mathbf{q}}} g$ . . . . .                          | 25        |
| 3.3.3. $\nabla_{\hat{\theta}} \hat{\mathbf{f}}$ . . . . .               | 26        |
| 3.4. Descenso de Gradiente para la Optimización de Parámetros . . . . . | 27        |
| 3.5. Algoritmo del Método Adjunto . . . . .                             | 31        |
| <b>4. Experimentos Numéricos</b>                                        | <b>33</b> |
| 4.1. Ecuación de Burgers Viscosa . . . . .                              | 34        |
| 4.2. Ecuación de Fisher-KPP . . . . .                                   | 43        |
| <b>5. Discusión</b>                                                     | <b>49</b> |

|                                                  |           |
|--------------------------------------------------|-----------|
| <b>6. Conclusiones</b>                           | <b>53</b> |
| <i>Referencias</i>                               | 55        |
| <b>Apéndices</b>                                 |           |
| <b>A. Fundamentos Teóricos Adicionales</b>       | <b>59</b> |
| A.1. Matricización de Tensores . . . . .         | 59        |
| A.2. Interpolación por Splines Cúbicos . . . . . | 61        |
| <b>B. Código en Python</b>                       | <b>63</b> |

# ÍNDICE DE FIGURAS

|                                                                                                                                                                                                           |    |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 3.1. Esquema del proceso de inferencia de modelos de orden reducido (ROM); flujo algebraico clásico de OpInf (rama superior) y refinamiento basado en el método adjunto dinámico (rama inferior). . . . . | 32 |
| 4.1. Dinámica de la ecuación de Burgers viscosa utilizada para la generación de datos. . . . .                                                                                                            | 35 |
| 4.2. Resultados de la reducción de la densidad de datos para el experimento sintético de la ecuación de Burgers. . . . .                                                                                  | 37 |
| 4.3. Error de predicción frente al tiempo para cada ejecución de reducción de densidad de datos en el experimento de la ecuación de Burgers. . . . .                                                      | 38 |
| 4.4. Error relativo frente a $r$ para cada ejecución de reducción de densidad de datos en el experimento de la ecuación de Burgers. . . . .                                                               | 39 |
| 4.5. Simulaciones de perturbación por ruido para el experimento sintético de la ecuación de Burgers. . . . .                                                                                              | 40 |
| 4.6. Error de predicción frente al tiempo para cada ejecución de nivel de ruido en el experimento de la ecuación de Burgers. . . . .                                                                      | 41 |
| 4.7. Error relativo frente a $r$ para cada ejecución de perturbación por ruido en el experimento de la ecuación de Burgers. . . . .                                                                       | 42 |
| 4.8. Dinámica de la ecuación FKPP utilizada para la generación de datos. . . . .                                                                                                                          | 44 |
| 4.9. Simulaciones de perturbación por ruido para el experimento sintético de la ecuación de FKPP. . . . .                                                                                                 | 45 |
| 4.10. Error de predicción frente al tiempo para cada ejecución de nivel de ruido en el experimento de la ecuación de FKPP. . . . .                                                                        | 46 |
| 4.11. Error relativo frente a $r$ para cada ejecución de perturbación por ruido en el experimento de la ecuación de FKPP. . . . .                                                                         | 47 |



# INTRODUCCIÓN

La modelización de orden reducido se sitúa en la intersección entre el análisis numérico y la computación científica basada en datos, ofreciendo técnicas potentes para la reducción de la complejidad en sistemas dinámicos. Esta tesis desarrolla un enfoque novedoso para el entrenamiento de modelos de orden reducido (ROMs, por sus siglas en inglés) robustos mediante la integración de la Inferencia de Operadores (OpInf) con el método del estado adjunto. Al tender un puente entre las técnicas numéricas clásicas y los paradigmas modernos de aprendizaje automático, presentamos un marco de trabajo que mejora la estabilidad y la robustez ante el ruido de los ROMs basados en datos, requisitos cruciales para aplicaciones en física computacional e ingeniería.

## Contexto Histórico

La modelización de orden reducido (ROM) cuenta con una larga trayectoria en el análisis numérico y la física computacional, habiendo surgido como un método para simplificar sistemas dinámicos complejos preservando al mismo tiempo sus características esenciales. Las primeras técnicas de reducción de modelos basadas en proyección se remontan al truncamiento balanceado y a la descomposición ortogonal propia (POD), las cuales se aplicaron ampliamente en la teoría de control y la dinámica de fluidos [1, 2]. Sin embargo, la idea de aprender modelos de orden reducido a partir de datos adquirió una importancia significativa en el siglo XXI, particularmente con el auge de las técnicas guiadas por datos y los nuevos enfoques de aprendizaje automático.

Un desarrollo notable en esta dirección es la Inferencia de Operadores (OpInf), un método introducido para aproximar operadores de orden reducido directamente a partir de datos de series temporales sin requerir acceso a las ecuaciones del sistema de orden completo. Este enfoque se ha formalizado como una técnica no intrusiva de reducción de modelos basada en proyección, lo que permite el aprendizaje eficiente de modelos de baja dimensión a partir de simulaciones de alta fidelidad [3]. De manera

más amplia, se ha explorado la integración de problemas inversos y la reducción de modelos, demostrando cómo se pueden inferir modelos basados en la física a partir de datos manteniendo la interpretabilidad y la consistencia física [4].

En años recientes, la intersección del aprendizaje automático con la modelización computacional ha dado lugar a marcos de trabajo novedosos tales como las Ecuaciones Diferenciales Ordinarias Neuronales (Neural ODEs) [5]. Estos modelos ofrecen una forma flexible de aprender dinámicas en tiempo continuo a partir de datos y han inspirado nuevas formulaciones para el aprendizaje de representaciones de orden reducido de sistemas dinámicos. Un aspecto crucial para entrenar tales modelos de manera eficiente es el cómputo de gradientes, el cual puede realizarse utilizando el método adjunto, un enfoque clásico en el control óptimo y en la optimización restringida por ecuaciones en derivadas parciales (EDPs) [6, 7].

Esta tesis se fundamenta en estos avances al entrenar la Inferencia de Operadores mediante métodos adjuntos. Específicamente, se tiene como objetivo reformular la derivación del problema adjunto dentro de la forma integral de OpInf. Al hacerlo, este trabajo busca desarrollar modelos de orden reducido robustos que puedan gestionar eficazmente datos imperfectos.

## Motivación del Estudio

La creciente complejidad de los modelos computacionales en física e ingeniería exige ROMs que mantengan la precisión al tiempo que logran una reducción dimensional significativa. Los métodos tradicionales de OpInf, aunque resultan eficaces en escenarios con datos limpios, enfrentan limitaciones fundamentales al lidiar con datos de medición del mundo real que contienen ruido u observaciones incompletas, o bien cuando la dinámica subyacente presenta fuertes no linealidades o dependencias complejas que son difíciles de representar mediante formas de operadores predefinidas. Este desafío motiva nuestra reformulación del problema de aprendizaje a través de un funcional de pérdida de base integral combinado con optimización basada en el estado adjunto.

Nuestro enfoque se dirige específicamente a problemas modelo canónicos en física computacional, tales como la ecuación de Burgers viscosa, que modela la formación de ondas de choque, y la ecuación de Fisher-KPP, que describe sistemas de reacción-difusión, donde las imperfecciones de los datos pueden afectar significativamente el rendimiento del modelo ROM. A diferencia de la OpInf convencional, que resuelve problemas de regresión lineal utilizando aproximaciones de derivadas potencialmente ruidosas, nuestro método evita la diferenciación finita explícita de los estados al reformular el problema mediante la forma integral de las ecuaciones gobernantes. Esta tesis investiga si la integración de la dinámica aprendida a lo largo del tiempo puede atenuar

intrínsecamente el ruido de medición, actuando como una suerte de filtro paso bajo integrado y, por ende, producir modelos de orden reducido más estables y confiables.

La innovación clave radica en combinar esta formulación integral con los métodos del estado adjunto para un cómputo eficiente del gradiente. Al derivar las ecuaciones adjuntas directamente para la forma integral de OpInf, hacemos posible la optimización basada en gradientes de los parámetros del ROM manteniendo la estabilidad numérica. Este enfoque ofrece las siguientes ventajas principales:

- (I) Supresión de la diferenciación numérica sobre datos de estado potencialmente ruidosos y dispersos,
- (II) Regularización inherente mediante la integración temporal,
- (III) Compatibilidad con marcos de diferenciación modernos (tales como las Neural ODEs) que capturan de forma natural relaciones complejas y no lineales en la dinámica del sistema,
- (IV) Eficiencia computacional y beneficios en el almacenamiento de memoria, dado que elimina la necesidad de almacenar información para las derivadas de los estados y, como se demostrará más adelante, el costo del método adjunto es constante con respecto al número de parámetros del modelo.

En conjunto, estas ventajas conducen a modelos significativamente más robustos, especialmente cuando se trabaja con datos imperfectos procedentes de mediciones experimentales y simulaciones.

Los objetivos principales del presente trabajo son:

- Derivar y reformular el problema adjunto en el contexto de la forma integral de OpInf, estableciendo los fundamentos teóricos para el cómputo de gradientes en el entrenamiento de ROMs de base integral.
- Desarrollar marcos de trabajo ROM que mantengan la estabilidad numérica durante la integración temporal, demostrando a su vez robustez ante diversas imperfecciones de los datos, incluyendo el ruido aditivo y la escasez de datos.

El resto de esta tesis se organiza de la siguiente manera:

- El Capítulo 2 introduce la Inferencia de Operadores, un ROM no intrusivo y guiado por datos que constituye la base del procedimiento de aprendizaje en el enfoque propuesto.
- El Capítulo 3 presenta una derivación detallada de las ecuaciones del método adjunto utilizadas para minimizar el funcional de pérdida en forma integral dentro del marco de OpInf.
- El Capítulo 4 reporta los experimentos numéricos que validan el método propuesto.

- Los Capítulos 5 y 6 discuten las implicaciones de los hallazgos, presentan las conclusiones y esbozan posibles líneas de investigación futura.

## PRELIMINARES

En este capítulo se introducen los conceptos fundamentales y las herramientas matemáticas que sustentan el desarrollo de los modelos de orden reducido presentados en esta tesis. Comenzamos con una visión general de los ROMs basados en proyección [2], destacando el papel de la Descomposición Ortogonal Propia (POD) [8] como una herramienta esencial para la extracción de modos dominantes a partir de conjuntos de datos de alta fidelidad. Posteriormente, se examinan los fundamentos teóricos de la Inferencia de Operadores [9].

### 2.1. Reducción de Orden de Modelos (MOR)

Un desafío de primer orden en muchos problemas de matemática aplicada, particularmente en aquellos gobernados por ecuaciones en derivadas parciales, radica en equilibrar la viabilidad computacional con una representación precisa de los fenómenos físicos subyacentes. En múltiples aplicaciones del análisis numérico y la física computacional, la dimensión  $n$  resulta extremadamente grande, lo que hace que las simulaciones directas sean computacionalmente costosas o incluso prohibitivas. El objetivo de la reducción de modelos es construir un modelo sustituto (o *modelo de orden reducido* – ROM) que aproxime con precisión la dinámica en un subespacio de dimensión mucho menor a través de técnicas de proyección.

Considérese un *modelo de orden completo* (FOM) de alta dimensión gobernado por un sistema de ecuaciones de evolución dependientes del tiempo:

$$\frac{d}{dt}\mathbf{q}(t) = \mathbf{f}(\mathbf{q}(t), \mathbf{u}(t)), \quad t \in [0, T], \quad (2.1)$$

donde  $\mathbf{q}(t) \in \mathbb{R}^n$  representa el vector de estado de un sistema complejo al tiempo  $t$ ,  $\mathbf{u}(t) \in \mathbb{R}^m$  denota las señales de entrada o términos de forzamiento, y la función  $\mathbf{f} : \mathbb{R} \times \mathbb{R}^n \times \mathbb{R}^m \rightarrow \mathbb{R}^n$  define la dinámica (posiblemente no lineal) derivada de una

discretización espacial de las ecuaciones gobernantes (frecuentemente EDPs).

Para fijar ideas, en esta tesis nos enfocaremos en el caso común donde la dinámica de orden completo admite una estructura de operadores cuadráticos. En otras palabras, asumimos que el miembro derecho de (2.1) se puede expresar como

$$\dot{\mathbf{q}}(t) = \mathbf{f}(\mathbf{q}(t), \mathbf{u}(t)) = \mathbf{c} + \mathbf{A}\mathbf{q}(t) + \mathbf{H}[\mathbf{q}(t) \otimes \mathbf{q}(t)] + \mathbf{B}\mathbf{u}(t), \quad (2.2)$$

donde  $\mathbf{c} \in \mathbb{R}^n$  es un vector constante,  $\mathbf{A} \in \mathbb{R}^{n \times n}$  y  $\mathbf{B} \in \mathbb{R}^{n \times m}$  son operadores lineales, y  $\mathbf{H} \in \mathbb{R}^{n \times n^2}$  captura las interacciones cuadráticas a través del producto tensorial  $\mathbf{q}(t) \otimes \mathbf{q}(t)$  (véase el Apéndice A.1 - Matricialización de tensores).

El ROM busca una aproximación precisa de la solución del FOM al restringir la dinámica a un subespacio reducido:

$$\mathbf{q}(t) \approx \mathbf{V}_r \hat{\mathbf{q}}(t), \quad \mathbf{V}_r \in \mathbb{R}^{n \times r}, \quad \hat{\mathbf{q}} \in \mathbb{R}^r, \quad r \ll n, \quad (2.3)$$

donde las columnas de  $\mathbf{V}_r$  forman una base reducida que genera dicho subespacio de baja dimensión.

Un enfoque común para la construcción de la base reducida es la descomposición ortogonal propia. Dada una colección de  $k$  instantáneas de la solución de (2.1), es decir,

$$\left. \frac{d}{dt} \mathbf{q}(t) \right|_{t=t_i} = \mathbf{f}(\mathbf{q}(t_i), \mathbf{u}(t_i))$$

en determinados puntos temporales  $t_i$  para  $i = 1, \dots, k$ , definimos la *matriz de instantáneas de estado*  $\mathbf{Q}$  como

$$\mathbf{Q} = \begin{bmatrix} | & | & & | \\ \mathbf{q}(t_1) & \mathbf{q}(t_2) & \cdots & \mathbf{q}(t_k) \\ | & | & & | \end{bmatrix}. \quad (2.4)$$

Entonces, la base POD (ortonormal)  $\mathbf{V}_r \in \mathbb{R}^{n \times r}$  se obtiene al minimizar el error de reconstrucción en el sentido de  $L^2$ :

$$\begin{aligned} \mathbf{V}_r = \operatorname{argmin}_{\mathbf{W} \in \mathbb{R}^{n \times r}} \quad & \sum_{i=1}^k \left\| \mathbf{q}(t_i) - \mathbf{W} \mathbf{W}^T \mathbf{q}(t_i) \right\|_2^2 \\ \text{s.t.} \quad & \mathbf{W}^T \mathbf{W} = \mathbf{I} \quad (\text{condición de Galerkin}). \end{aligned} \quad (2.5)$$

Esta formulación es equivalente a computar una descomposición en valores singulares (SVD) truncada de rango  $r$  de la matriz de instantáneas de estado  $\mathbf{Q}$ ,

$$\mathbf{Q} = \Phi \Sigma \Psi^T,$$

donde los vectores de la base POD están dados por las primeras  $r$  columnas de  $\Phi$ :

$$\mathbf{V}_r = \Phi_{:,1:r}. \quad (2.6)$$

La selección de la dimensión de truncamiento  $r$  se basa típicamente en el contenido de energía relativa de los valores singulares (energía acumulada,  $\kappa_r$ ):

$$\kappa_r = \frac{\sum_{i=1}^r \sigma_i^2}{\sum_{i=1}^k \sigma_i^2} \geq 1 - \epsilon_{\text{POD}}, \quad (2.7)$$

donde  $\epsilon_{\text{POD}} > 0$  es una tolerancia prescrita y  $\sigma_i$  denota el  $i$ -ésimo valor singular de  $\mathbf{Q}$ . La optimalidad de la base POD se caracteriza por el error de proyección:

$$\sum_{i=1}^k \left\| \mathbf{q}(t_i) - \mathbf{V}_r \mathbf{V}_r^T \mathbf{q}(t_i) \right\|_2^2 = \sum_{i=r+1}^k \sigma_i^2, \quad (2.8)$$

el cual se minimiza entre todas las bases ortonormales de rango  $r$ .

La proyección de la solución de orden completo sobre el subespacio generado por  $\mathbf{V}_r$  proporciona una representación reducida que captura los modos más energéticos del sistema, dando lugar a lo que se conoce como el *sistema de dinámica reducida*

$$\dot{\hat{\mathbf{q}}}(t) = \mathbf{V}_r^T \mathbf{V}_r \dot{\mathbf{q}}(t) = \mathbf{V}_r^T \dot{\mathbf{q}}(t) = \mathbf{V}_r^T \mathbf{f}(\mathbf{V}_r \hat{\mathbf{q}}(t), \mathbf{u}(t)). \quad (2.9)$$

Considérese el sistema de alta dimensión en (2.2). Expandiendo la dinámica en forma polinomial, se tiene

$$\begin{aligned} \dot{\hat{\mathbf{q}}}(t) &= \mathbf{V}_r^T \mathbf{f}(\mathbf{V}_r \hat{\mathbf{q}}(t), \mathbf{u}(t)) \\ &= \mathbf{V}_r^T (\mathbf{c} + \mathbf{A} \mathbf{V}_r \hat{\mathbf{q}}(t) + \mathbf{H}[(\mathbf{V}_r \hat{\mathbf{q}}(t)) \otimes (\mathbf{V}_r \hat{\mathbf{q}}(t))] + \mathbf{B} \mathbf{u}(t)). \end{aligned}$$

Dado que  $\mathbf{V}_r^T \mathbf{V}_r = \mathbf{I}$  y utilizando la propiedad  $(\mathbf{X}\mathbf{Y}) \otimes (\mathbf{Z}\mathbf{W}) = (\mathbf{X} \otimes \mathbf{Z})(\mathbf{Y} \otimes \mathbf{W})$ , la expresión anterior se puede simplificar como

$$\begin{aligned} \dot{\hat{\mathbf{q}}}(t) &= \mathbf{V}_r^T \mathbf{f}(t, \mathbf{V}_r \hat{\mathbf{q}}(t), \mathbf{u}(t)) = \hat{\mathbf{c}} + \hat{\mathbf{A}} \hat{\mathbf{q}}(t) + \hat{\mathbf{H}}[\hat{\mathbf{q}}(t) \otimes \hat{\mathbf{q}}(t)] + \hat{\mathbf{B}} \mathbf{u}(t), \\ &=: \hat{\mathbf{f}}(\hat{\mathbf{q}}(t), \mathbf{u}(t); \hat{\theta}) \end{aligned} \quad (2.10)$$

donde los *operadores de orden reducido* se definen como

$$\mathbb{R}^r \ni \hat{\mathbf{c}} = \mathbf{V}_r^T \mathbf{c}, \quad \mathbb{R}^{r \times r} \ni \hat{\mathbf{A}} = \mathbf{V}_r^T \mathbf{A} \mathbf{V}_r, \quad \mathbb{R}^{r \times r^2} \ni \hat{\mathbf{H}} = \mathbf{V}_r^T \mathbf{H}(\mathbf{V}_r \otimes \mathbf{V}_r^T), \quad \mathbb{R}^{r \times m} \ni \hat{\mathbf{B}} = \mathbf{V}_r^T \mathbf{B},$$

$\hat{\theta} = [\hat{\mathbf{c}} \ \hat{\mathbf{A}} \ \hat{\mathbf{H}} \ \hat{\mathbf{B}}] \in \hat{\Theta}$  denota la colección de parámetros donde  $\hat{\Theta} \in \mathbb{R}^{r \times (1+r+r^2+m)}$  representa el espacio de parámetros admisible, y  $\hat{\mathbf{f}}$  es el miembro derecho del modelo

reducido. Por lo tanto, el ROM de Galerkin basado en proyección preserva la misma estructura polinomial que  $\mathbf{f}$  (2.2). Más adelante en este trabajo, consideraremos la forma más simple  $\hat{\mathbf{f}}(\hat{\mathbf{q}}(t), \hat{\theta})$ , omitiendo la entrada dada  $\mathbf{u}$  como variable, dado que no resulta relevante en la derivación de nuestro nuevo marco de trabajo.

Dentro de MOR diferenciamos entre:

- *Enfoque intrusivo (basado en proyección)*: Requiere acceso explícito a  $\mathbf{c}$ ,  $\mathbf{A}$ ,  $\mathbf{H}$  y  $\mathbf{B}$  para poder ensamblar los operadores de orden reducido. Esto puede resultar complejo si el solver de alta fidelidad es propietario o funciona como una caja negra.
- *Enfoque no intrusivo (guiado por datos)*: No proyecta explícitamente  $\mathbf{c}$ ,  $\mathbf{A}$ ,  $\mathbf{H}$  y  $\mathbf{B}$ . En su lugar, se recopilan datos de series temporales (instantáneas de  $\mathbf{q}(t)$  y, si es necesario, de  $\dot{\mathbf{q}}(t)$ ), se proyectan sobre  $\mathbf{V}_r$  y se aprenden los operadores reducidos mediante regresión. Este enfoque resulta ideal si el solver original es una caja negra o si su modificación es prohibitivamente compleja. La Inferencia de Operadores es un método no intrusivo prototípico.

En ambos casos, el enfoque MOR produce un modelo sustituto eficiente del sistema dinámico de orden completo que reside en un subespacio de dimensión significativamente menor. Como resultado, hace posible simulaciones y análisis más rápidos al tiempo que conserva las características físicas esenciales del sistema original.

Se pueden encontrar más detalles sobre las técnicas de proyección y los aspectos prácticos de implementación de los ROMs basados en POD en [9].

## 2.2. Inferencia de Operadores (OpInf)

La Inferencia de Operadores (OpInf) es una metodología de reducción de modelos basada en datos que combina aspectos clave de la reducción de modelos por proyección con técnicas de aprendizaje guiadas por datos [3, 10]. Este enfoque permite la construcción de modelos de orden reducido computacionalmente eficientes, manteniendo al mismo tiempo la interpretabilidad física mediante la preservación de la estructura de las ecuaciones gobernantes.

### Descripción General de la Metodología

El concepto fundamental de OpInf radica en inferir los operadores de un modelo de orden reducido mediante un proceso de aprendizaje que aprovecha tanto los datos de simulación de alta fidelidad como la estructura subyacente de las ecuaciones gobernantes. El método opera a través de tres etapas principales:

1. **Recopilación de Datos:** Reunir las instantáneas de la solución  $\{\mathbf{q}(t_i)\}_{i=1}^k$  y las entradas correspondientes a partir de simulaciones numéricas de alta fidelidad del modelo de orden completo.
2. **Proyección de Estados:** Emplear técnicas de reducción de la dimensionalidad, tales como la descomposición ortogonal propia, para identificar un subespacio de baja dimensión que capture la dinámica esencial. Esto proyecta el estado de alta dimensión  $\mathbf{q}(t) \in \mathbb{R}^n$  a un estado reducido  $\hat{\mathbf{q}}(t) \in \mathbb{R}^r$ , donde  $r \ll n$ .
3. **Inferencia de Operadores:** Determinar los operadores de orden reducido mediante una optimización por mínimos cuadrados que obliga al modelo aprendido a alinearse tanto con los datos proyectados como con la estructura de las ecuaciones gobernantes.

### Formulación Matemática

En su esencia, OpInf asume una forma polinomial para la dinámica reducida en el subespacio de dimensión  $r$  y resuelve un problema de mínimos cuadrados para inferir los operadores reducidos a partir de los datos mediante los siguientes pasos:

- 1) **Recopilación de datos.** Simular o experimentar con el modelo completo (2.6) bajo uno o más escenarios de entrada  $\mathbf{u}(t)$ , y almacenar:

$$\{\mathbf{q}(t_i)\}_{i=1}^k \quad \text{y} \quad \{\dot{\mathbf{q}}(t_i)\}_{i=1}^k,$$

si están disponibles directamente o aproximados por diferencias finitas.

- 2) **Base reducida mediante POD y proyección a coordenadas reducidas.** Formar la matriz de instantáneas

$$\mathbf{Q} = [\mathbf{q}(t_1) \quad \mathbf{q}(t_2) \quad \cdots \quad \mathbf{q}(t_k)] \in \mathbb{R}^{n \times k}.$$

Computar su descomposición en valores singulares truncada para obtener  $\mathbf{V}_r$  como en (2.7), donde  $r \ll n$  se selecciona con base en el decaimiento de los valores singulares o en un umbral de energía. Luego, para cada instantánea,

$$\hat{\mathbf{q}}(t_i) = \mathbf{V}^T \mathbf{q}(t_i) \in \mathbb{R}^r, \quad \dot{\hat{\mathbf{q}}}(t_i) = \mathbf{V}^T \dot{\mathbf{q}}(t_i) \in \mathbb{R}^r.$$

- 3) **Asumir una forma de orden reducido y formular un problema de regresión (mínimos cuadrados).** Prescribir una estructura para la ODE de dimensión  $r$ . Una elección común incluye términos constantes, lineales, cuadráticos y de control, por lo que definimos la función de pérdida de entrenamiento discreta como

$$\sum_{i=1}^k \left\| [\hat{\mathbf{c}} + \hat{\mathbf{A}}\hat{\mathbf{q}}(t_i) + \hat{\mathbf{H}}[\hat{\mathbf{q}}(t_i) \otimes \hat{\mathbf{q}}(t_i)] + \hat{\mathbf{B}}\mathbf{u}(t_i)] - \dot{\hat{\mathbf{q}}}(t_i) \right\|_2^2, \quad (2.11)$$

donde los operadores reducidos  $\hat{\mathbf{c}}$ ,  $\hat{\mathbf{A}}$ ,  $\hat{\mathbf{H}}$  y  $\hat{\mathbf{B}}$  son los elementos a inferir.

Finalmente, OpInf identifica los operadores reducidos resolviendo el siguiente problema de mínimos cuadrados (una regresión lineal en  $\hat{\theta}$ ) para minimizar la pérdida de entrenamiento

$$\min_{\hat{\theta} \in \hat{\Theta}} \sum_{i=1}^k \left\| [\hat{\mathbf{c}} + \hat{\mathbf{A}}\hat{\mathbf{q}}(t_i) + \hat{\mathbf{H}}[\hat{\mathbf{q}}(t_i) \otimes \hat{\mathbf{q}}(t_i)] + \hat{\mathbf{B}}\mathbf{u}(t_i)] - \hat{\mathbf{q}}(t_i) \right\|_2^2. \quad (2.12)$$

El uso de parametrizaciones polinomiales en la Inferencia de Operadores se justifica por el hecho de que muchos modelos de orden reducido basados en proyección de EDPs adoptan de manera natural una forma polinomial. Por ejemplo, la semidiscretización de la ecuación de Burgers viscosa conduce típicamente a un sistema de ODEs de orden completo con no linealidades cuadráticas. Una posterior proyección de Galerkin sobre una base de descomposición ortogonal propia (POD) produce entonces un sistema cuadrático reducido.

En la reducción de modelos no intrusiva, tal como OpInf, no calculamos los operadores reducidos mediante la proyección de Galerkin, ya que esto requeriría acceder a los componentes internos del solver de orden completo. En su lugar, los operadores reducidos se infieren directamente a partir de los datos. Este enfoque asume que los datos de entrenamiento reflejan una estructura polinomial compatible con la forma del modelo seleccionado.

Cuando el sistema original involucra no linealidades no polinomiales, este supuesto puede fallar, lo que resulta en un rendimiento deficiente del modelo. En tales casos, se puede aplicar una transformación de elevación (*lifting transformation*) para reescribir el sistema en un espacio de mayor dimensión donde la dinámica exhiba una estructura polinomial. Estos mapas de elevación, que a menudo introducen variables auxiliares, permiten que la Inferencia de Operadores aproxime el sistema con mayor precisión al alinear los datos con la estructura polinomial del modelo.

En la práctica, también es habitual incluir un término de regularización en la pérdida de entrenamiento,  $\mathcal{R}(\hat{\theta})$  en (2.12), el cual penaliza la magnitud de las entradas de los operadores aprendidos. Esta regularización mejora el condicionamiento del problema de aprendizaje. Para mayores detalles e implementaciones numéricas, se remite al lector al recurso en línea [9] y al trabajo fundamental de Peherstorfer y Willcox [3].

## Ventajas y Aplicaciones

La Inferencia de Operadores ofrece diversas ventajas distintivas en comparación con los enfoques tradicionales de reducción de modelos:

- **No intrusiva:** Requiere únicamente instantáneas de la solución en lugar de un acceso explícito a los operadores del modelo de orden completo.
- **Consciente de la física (*physics-aware*):** La proyección de Galerkin preserva la estructura polinomial de las ecuaciones gobernantes de orden completo en el sistema reducido. En efecto, una comparación término a término de (2.2) y (2.10) confirma esta correspondencia biunívoca.
- **Computacionalmente eficiente:** Evita operaciones costosas en alta dimensión mediante la reducción de la dimensionalidad.

Esta metodología se ha aplicado con éxito a diversos sistemas dinámicos complejos, incluyendo flujos de fluidos, sistemas térmicos y dinámica de estructuras [4, 11]. La combinación del aprendizaje guiado por datos con restricciones basadas en la física hace que la Inferencia de Operadores sea particularmente eficaz para sistemas en los que los métodos tradicionales basados en proyección se vuelven computacionalmente prohibitivos o donde los operadores del modelo de orden completo no están explícitamente disponibles. A pesar de estas fortalezas, una limitación clave de OpInf es su dependencia del cómputo preciso de las derivadas temporales a partir de los datos, lo cual puede verse significativamente comprometido cuando los datos disponibles son imperfectos o ruidosos. En el próximo capítulo, introducimos un enfoque novedoso diseñado para superar este desafío, mejorando así la robustez y la precisión de los operadores reducidos inferidos.



# MÉTODO ADJUNTO PARA LA INFERENCIA DE OPERADORES

## 3.1. Un Funcional de Pérdida en Tiempo Continuo

Para la derivación de este nuevo marco de trabajo, se asume una trayectoria continua  $\hat{\mathbf{q}}(t) \in \mathcal{C}^1(\mathcal{T})^r$  sobre un dominio temporal  $\mathcal{T} = [0, T]$ . Sin embargo, en la práctica, únicamente disponemos de instantáneas temporales discretas  $\{(t_i, \mathbf{q}_{\text{true}}(t_i))\}_{i=1}^k$ . Para realizar la transición del tiempo discreto al continuo, aproximamos la función de suma de costos  $g$  mediante interpolación (véase el Apéndice A.2 — Splines Cúbicos) [12]:

$$\sum_{i=1}^k \underbrace{\|\hat{\mathbf{q}}_{\text{true}}(t_i) - \tilde{\mathbf{q}}(t_i, \hat{\theta})\|_2^2}_{g(\tilde{\mathbf{q}}(t_i, \hat{\theta}), t_i)} \Delta t \xrightarrow{\text{interp.}} \int_0^T \underbrace{\|\hat{\mathbf{q}}_{\text{true}}(t) - \tilde{\mathbf{q}}(t, \hat{\theta})\|_2^2}_{g(\tilde{\mathbf{q}}(t, \hat{\theta}), t)} dt,$$

donde  $\tilde{\mathbf{q}}(t, \hat{\theta})$  denota la trayectoria predicha reducida producida por nuestro modelo dinámico (es decir, la solución de (2.10)). Puesto que  $\tilde{\mathbf{q}}(t, \hat{\theta})$  se genera a partir de las instantáneas discretas de referencia (“ground-truth”)  $\{\mathbf{q}_{\text{true}}(t_i)\}$ , solo requerimos interpolar dichos valores de  $\mathbf{q}_{\text{true}}(t_i)$  en el tiempo.

Con el fin de evitar la diferenciación explícita que caracteriza al enfoque estándar de OpInf, introducemos un funcional de pérdida en tiempo continuo que compara, en el sentido de  $L^2$ , la trayectoria reconstruida mediante la integración de los operadores aprendidos frente a los datos observados.

**Definición 3.1.1** *Considérese el miembro derecho del sistema reducido, es decir,  $\hat{\mathbf{f}}(\hat{\mathbf{q}}(t), \hat{\theta})$ , como en (2.10), para una entrada dada  $\mathbf{u}(t)$  sobre el dominio temporal  $\mathcal{T} = [0, T]$ . Sean  $\hat{\mathbf{q}} \in \mathcal{C}^1(\mathcal{T})^r$  la trayectoria reducida y  $[\hat{\mathbf{c}} \ \hat{\mathbf{A}} \ \hat{\mathbf{H}} \ \hat{\mathbf{B}}] = \hat{\theta} \in \mathbb{R}^d$  el vector de todos los operadores reducidos, con un total de  $d = r + r^2 + r^3 + rm$  parámetros. Definimos el funcional de*

pérdida de entrenamiento en tiempo continuo  $\ell(\hat{\mathbf{q}}(t))$  como la discrepancia entre los datos observados y los predichos, es decir,

$$\ell : \mathcal{C}^1(\mathcal{T})^r \rightarrow \mathbb{R}, \quad \hat{\mathbf{q}}(\cdot) \mapsto \int_0^T \left\| \hat{\mathbf{q}}(t) - \hat{\mathbf{q}}_{\text{true}}(t) \right\|_2^2 dt = \int_0^T g(\hat{\mathbf{q}}(t), t) dt, \quad (3.1)$$

donde  $g(\hat{\mathbf{q}}(t), t) := \left\| \hat{\mathbf{q}}(t) - \hat{\mathbf{q}}_{\text{true}}(t) \right\|_2^2$ . Definimos el funcional de pérdida reducido como

$$\tilde{\ell} : \mathbb{R}^d \rightarrow \mathbb{R}, \quad \hat{\theta} \mapsto \int_0^T \left\| \tilde{\mathbf{q}}(t, \hat{\theta}) - \hat{\mathbf{q}}_{\text{true}}(t) \right\|_2^2 dt, \quad (3.2)$$

donde

$$\begin{aligned} \tilde{\mathbf{q}}(t, \hat{\theta}) &= \tilde{\mathbf{q}}(0) + \int_0^t \hat{\mathbf{f}}(\tilde{\mathbf{q}}(\tau), \hat{\theta}) d\tau \\ &= \tilde{\mathbf{q}}(0) + \int_0^t [\hat{\mathbf{c}} + \hat{\mathbf{A}}\tilde{\mathbf{q}}(\tau) + \hat{\mathbf{H}}(\tilde{\mathbf{q}}(\tau) \otimes \tilde{\mathbf{q}}(\tau)) + \hat{\mathbf{B}}\mathbf{u}(\tau)] d\tau, \quad t \in \mathcal{T} \end{aligned} \quad (3.3)$$

es la solución de

$$\dot{\tilde{\mathbf{q}}}(t) = \hat{\mathbf{f}}(\tilde{\mathbf{q}}(t), \hat{\theta}), \quad \tilde{\mathbf{q}}(0) = \hat{\mathbf{q}}_0, \quad (3.4)$$

para un  $\hat{\theta}$  dado.

Obsérvese que  $\tilde{\ell}(\hat{\theta}) = \ell(\tilde{\mathbf{q}}(\cdot, \hat{\theta}))$  dado que  $\tilde{\mathbf{q}}$  satisface la ecuación gobernante de orden reducido (3.4).

Con base en el funcional de pérdida reducido (3.2) que considera la integral temporal del miembro derecho de nuestro sistema dinámico reducido, la tarea de aprendizaje consiste en hallar

$$\hat{\theta}^* = \min_{\hat{\theta}} \tilde{\ell}(\hat{\theta}). \quad (3.5)$$

Para diferenciar funcionales, recordamos en primer lugar la derivada de Gâteaux [13]:

**Definición 3.1.2** Para una aplicación  $\mathbf{f} : \mathcal{X} \rightarrow \mathcal{Y}$  entre espacios normados, la derivada de Gâteaux de  $\mathbf{f}$  en  $\mathbf{x}$  en la “dirección”  $\mathbf{v}$  se define como

$$\mathbf{D}_{\mathbf{x}} \mathbf{f}(\mathbf{x}; \mathbf{v}) := \lim_{\varepsilon \rightarrow 0} \frac{\mathbf{f}(\mathbf{x} + \varepsilon \mathbf{v}) - \mathbf{f}(\mathbf{x})}{\varepsilon},$$

siempre que dicho límite exista.

Cabe señalar que el subíndice  $\mathbf{x}$  denota la diferenciación respecto a esta variable, en particular cuando  $\hat{\mathbf{f}}$  depende de múltiples variables. Para minimizar el funcional de

pérdida reducido, calculamos el gradiente total con respecto a los parámetros. Por la regla de la cadena tenemos:

$$\frac{d\tilde{\ell}(\hat{\theta})}{d\hat{\theta}} = \frac{d\ell(\hat{\mathbf{q}}(\cdot, \hat{\theta}))}{d\hat{\theta}} = \underbrace{\frac{\partial \ell}{\partial \hat{\theta}}}_{\text{dependencia directa} = 0} + \underbrace{\frac{\partial \ell}{\partial \hat{\mathbf{q}}} \bigg|_{\hat{\mathbf{q}}(\cdot) = \hat{\mathbf{q}}(\cdot, \hat{\theta})}}_{\text{dependencia implícita}} \cdot \frac{d\hat{\mathbf{q}}}{d\hat{\theta}}, \quad (3.6)$$

donde el término  $d\hat{\mathbf{q}}/d\hat{\theta}$  codifica el modo en que la trayectoria reducida cambia ante variaciones en los parámetros. Debido a que  $\ell$  no posee una dependencia directa de  $\hat{\theta}$ , el primer término se anula. Reconocemos ahora que

$$\frac{\partial \ell}{\partial \hat{\mathbf{q}}} \bigg|_{\hat{\mathbf{q}}(\cdot) = \hat{\mathbf{q}}(\cdot, \hat{\theta})}$$

es la derivada de un funcional con respecto a una función. Utilizando la notación introducida en la 3.1.2, escribimos

$$\frac{\partial \ell}{\partial \hat{\mathbf{q}}} \bigg|_{\hat{\mathbf{q}}(\cdot) = \hat{\mathbf{q}}(\cdot, \hat{\theta})} \cdot \frac{d\hat{\mathbf{q}}}{d\hat{\theta}} = \mathbf{D} \ell \left( \hat{\mathbf{q}}(\cdot, \hat{\theta}); \frac{d\hat{\mathbf{q}}(\cdot, \hat{\theta})}{d\hat{\theta}} \right).$$

El cómputo directo de esta sensibilidad  $d\hat{\mathbf{q}}/d\hat{\theta}$  es costoso y propenso a sufrir inestabilidad numérica. Para eludir esta dificultad, empleamos el método adjunto, el cual reformula el gradiente en términos de una variable adjunta que satisface una ecuación diferencial hacia atrás en el tiempo (*backward-in-time*). En la Figura 3.1 se ilustra una descripción general del proceso de minimización (OpInf frente al método adjunto).

## 3.2. Las Ecuaciones del Estado Adjunto

### Configuración del Problema

Para una colección de datos de instantáneas dada, consideramos sus correspondientes datos reales continuos interpolados  $\hat{\mathbf{q}}_{\text{true}}(\cdot)$ . Planteamos la minimización de la pérdida como un problema de optimización con restricciones de igualdad siguiendo el enfoque de [7], es decir:

$$\begin{aligned} \min_{\hat{\theta}} \tilde{\ell}(\hat{\theta}) &\equiv \min_{\hat{\theta}, \hat{\mathbf{q}}(\cdot)} \ell(\hat{\mathbf{q}}(\cdot)) \\ \text{s.t. } \dot{\hat{\mathbf{q}}}(t) &= \hat{\mathbf{f}}(\hat{\mathbf{q}}(t), \hat{\theta}), \quad \hat{\mathbf{q}}(0) = \hat{\mathbf{q}}_0, \quad t \in [0, T], \end{aligned} \quad (3.7)$$

donde  $\ell$ ,  $\tilde{\ell}$  son los funcionales de pérdida definidos en (3.1) y (3.2), y la restricción representa el sistema ROM-OpInf descrito en (2.10).

Obsérvese que en la minimización conjunta (3.7), el funcional de pérdida  $\ell$  de-

pende exclusivamente de la trayectoria  $\hat{\mathbf{q}}(\cdot)$ , midiendo el desajuste respecto a los datos reales  $\hat{\mathbf{q}}_{\text{true}}(\cdot)$ , y no depende de manera explícita de  $\hat{\theta}$ . Por consiguiente, al escribir

$$\min_{\hat{\theta}, \hat{\mathbf{q}}(\cdot)} \ell(\hat{\mathbf{q}}(\cdot)),$$

se sobreentiende que  $\ell$  se evalúa únicamente sobre  $\hat{\mathbf{q}}(\cdot)$ , mientras que  $\hat{\theta}$  interviene exclusivamente a través de la restricción

$$\dot{\hat{\mathbf{q}}}(t) = \hat{\mathbf{f}}(\hat{\mathbf{q}}(t), \hat{\theta}).$$

El Lagrangiano asociado a (3.7) se define como

$$\mathcal{L}(\hat{\mathbf{q}}(\cdot), \hat{\theta}, \lambda(\cdot)) := \ell(\hat{\mathbf{q}}(\cdot)) - \int_0^T \lambda(t)^\top (\dot{\hat{\mathbf{q}}}(t) - \hat{\mathbf{f}}(\hat{\mathbf{q}}(t), \hat{\theta})) dt - \lambda(0)^\top (\hat{\mathbf{q}}(0) - \mathbf{q}_0). \quad (3.8)$$

Dado que las restricciones se satisfacen exactamente para la trayectoria óptima  $\tilde{\mathbf{q}}(\cdot, \hat{\theta})$ , se deduce que

$$\mathcal{L}(\tilde{\mathbf{q}}(\cdot, \hat{\theta}), \hat{\theta}, \lambda(\cdot)) = \tilde{\ell}(\hat{\theta}), \quad (3.9)$$

lo cual es válido para todo  $\lambda(\cdot) \in \mathcal{C}^1([0, T])^r$ .

Con el propósito de ejecutar un proceso de optimización basado en gradientes, se requiere calcular

$$\begin{aligned} \frac{d\tilde{\ell}(\hat{\theta})}{d\hat{\theta}} &= \frac{d\mathcal{L}}{d\hat{\theta}}(\tilde{\mathbf{q}}(\cdot, \hat{\theta}), \hat{\theta}, \lambda(\cdot)) \\ &= \frac{\partial \mathcal{L}}{\partial \hat{\theta}} + \frac{\partial \mathcal{L}}{\partial \hat{\mathbf{q}}} \bigg|_{\hat{\mathbf{q}}(\cdot) = \tilde{\mathbf{q}}(\cdot, \hat{\theta})} \cdot \frac{d\tilde{\mathbf{q}}}{d\hat{\theta}} \\ &= \frac{\partial \mathcal{L}}{\partial \hat{\theta}} + \mathbf{D}_{\hat{\mathbf{q}}} \mathcal{L} \left( \tilde{\mathbf{q}}(\cdot, \hat{\theta}), \hat{\theta}, \lambda(\cdot); \frac{d\tilde{\mathbf{q}}(\cdot, \hat{\theta})}{d\hat{\theta}} \right). \end{aligned} \quad (3.10)$$

A continuación, imponemos la condición de estacionariedad de la derivada,

$$\frac{\partial \mathcal{L}}{\partial \hat{\mathbf{q}}} \bigg|_{\hat{\mathbf{q}}(\cdot) = \tilde{\mathbf{q}}(\cdot, \hat{\theta})} = \mathbf{0}.$$

Esta condición proveerá las ecuaciones adjuntas necesarias para determinar  $\lambda$ , asegurando que

$$\frac{d\tilde{\ell}(\hat{\theta})}{d\hat{\theta}} = \frac{\partial \mathcal{L}}{\partial \hat{\theta}}(\tilde{\mathbf{q}}(\cdot, \hat{\theta}), \hat{\theta}, \lambda(\cdot)). \quad (3.11)$$

Para dicho problema, el método adjunto clásico tal como se formula en [7, 14] proporciona el siguiente resultado.

**Teorema 3.2.1 (Método adjunto)** *Asúmase que  $\hat{\mathbf{f}}$  y  $g$  son continuamente diferenciables tanto en  $\hat{\mathbf{q}}$  como en  $\hat{\theta}$ , y que el estado inicial  $\hat{\mathbf{q}}(0) = \hat{\mathbf{q}}_0$  es fijo (independiente de  $\hat{\theta}$ ). La variable adjunta  $\lambda(t) \in \mathcal{C}^1(\mathcal{T})^r$  asociada a un problema de minimización como el de (3.7) satisface una ecuación diferencial ordinaria (EDO) de retropropagación*

$$\dot{\lambda}(t) = - \left[ \left( \frac{\partial \hat{\mathbf{f}}}{\partial \hat{\mathbf{q}}} \right)^\top \lambda(t) + \left( \frac{\partial g}{\partial \hat{\mathbf{q}}} \right)^\top \right] \Big|_{\hat{\mathbf{q}}(\cdot) = \tilde{\mathbf{q}}(\cdot, \hat{\theta})}, \quad \lambda(T) = \mathbf{0}. \quad (3.12)$$

Además, el gradiente de  $\tilde{\ell}$  con respecto a los parámetros para la pérdida de entrenamiento en (3.2) adopta la forma

$$\frac{d\tilde{\ell}(\hat{\theta})}{d\hat{\theta}} = \int_0^T \lambda(t)^\top \frac{\partial \hat{\mathbf{f}}(\hat{\mathbf{q}}, \hat{\theta})}{\partial \hat{\theta}} \Big|_{\hat{\mathbf{q}}(\cdot) = \tilde{\mathbf{q}}(\cdot, \hat{\theta})} dt, \quad (3.13)$$

donde  $\lambda$  es la solución de (3.12).

### Demostración (Formulación Lagrangiana)

El Teorema (3.2.1) y su demostración se inspiraron en las ideas y ejemplos presentados en [6, 7], adaptando los pasos clave esenciales a nuestro problema y supuestos.

Considérese el Lagrangiano ( $\mathcal{L}$ ) definido como

$$\begin{aligned} \mathcal{L}(\tilde{\mathbf{q}}(\cdot, \hat{\theta}), \hat{\theta}, \lambda(\cdot)) &:= \ell(\tilde{\mathbf{q}}(t, \hat{\theta})) - \int_0^T \lambda(t)^\top (\dot{\tilde{\mathbf{q}}}(t, \hat{\theta}) - \hat{\mathbf{f}}(\tilde{\mathbf{q}}(t, \hat{\theta}), \hat{\theta})) dt - \lambda(0)^\top (\tilde{\mathbf{q}}(0, \hat{\theta}) - \tilde{\mathbf{q}}_0) \\ &= \int_0^T (g(\tilde{\mathbf{q}}(t, \hat{\theta}), t) - \lambda(t)^\top (\dot{\tilde{\mathbf{q}}}(t) - \hat{\mathbf{f}}(\tilde{\mathbf{q}}(t, \hat{\theta}), \hat{\theta}))) dt - \lambda(0)^\top (\tilde{\mathbf{q}}(0, \hat{\theta}) - \tilde{\mathbf{q}}_0). \end{aligned}$$

Diferenciando el Lagrangiano con respecto a  $\hat{\theta}$ , obtenemos

$$\begin{aligned} \frac{d\mathcal{L}}{d\hat{\theta}}(\tilde{\mathbf{q}}(\cdot, \hat{\theta}), \hat{\theta}, \lambda(\cdot)) &= \int_0^T \left[ \frac{\partial g}{\partial \hat{\theta}} + \frac{\partial g}{\partial \tilde{\mathbf{q}}} \frac{d\tilde{\mathbf{q}}}{d\hat{\theta}} - \lambda(t)^\top \left( \frac{d}{dt} \frac{d\tilde{\mathbf{q}}}{d\hat{\theta}} - \frac{\partial \hat{\mathbf{f}}}{\partial \hat{\theta}} - \frac{\partial \hat{\mathbf{f}}}{\partial \tilde{\mathbf{q}}} \frac{d\tilde{\mathbf{q}}}{d\hat{\theta}} \right) \right] dt - \lambda(0)^\top \frac{d\tilde{\mathbf{q}}}{d\hat{\theta}}(0) \\ &= \int_0^T \left[ \frac{\partial g}{\partial \hat{\theta}} + \lambda(t)^\top \frac{\partial \hat{\mathbf{f}}}{\partial \hat{\theta}} + \left( \frac{\partial g}{\partial \tilde{\mathbf{q}}} + \lambda(t)^\top \frac{\partial \hat{\mathbf{f}}}{\partial \tilde{\mathbf{q}}} - \lambda(t)^\top \frac{d}{dt} \right) \frac{d\tilde{\mathbf{q}}}{d\hat{\theta}} \right] dt - \lambda(0)^\top \frac{d\tilde{\mathbf{q}}}{d\hat{\theta}}(0). \end{aligned}$$

Con el propósito de evitar el cálculo de  $d\tilde{\mathbf{q}}/d\hat{\theta}$  (es decir, la sensibilidad de la trayectoria respecto a los parámetros, cuya obtención puede ser extremadamente compleja), aplicamos un cambio de variables mediante el método adjunto. Integrando por partes, eliminamos el término  $\frac{d}{dt} \frac{d\tilde{\mathbf{q}}}{d\hat{\theta}}$  en la última porción de la integral anterior:

$$\begin{aligned}
\int_0^T -\lambda(t)^\top \frac{d}{dt} \frac{d\tilde{\mathbf{q}}}{d\hat{\theta}} dt &= \left[ -\lambda(t)^\top \frac{d\tilde{\mathbf{q}}}{d\hat{\theta}} \right]_0^T + \int_0^T \left( \frac{d\lambda}{dt} \right)^\top \frac{d\tilde{\mathbf{q}}}{d\hat{\theta}} dt \\
&= \lambda(0)^\top \frac{d\tilde{\mathbf{q}}}{d\hat{\theta}}(0) - \lambda(T)^\top \frac{d\tilde{\mathbf{q}}}{d\hat{\theta}}(T) + \int_0^T \left( \frac{d\lambda}{dt} \right)^\top \frac{d\tilde{\mathbf{q}}}{d\hat{\theta}} dt.
\end{aligned}$$

Al sustituir esta expresión, la derivada del Lagrangiano resulta en

$$\begin{aligned}
\frac{d\mathcal{L}}{d\hat{\theta}}(\tilde{\mathbf{q}}(\cdot, \hat{\theta}), \hat{\theta}, \lambda(\cdot)) &= \int_0^T \left[ \frac{\partial g}{\partial \hat{\theta}} + \lambda(t)^\top \frac{\partial \hat{\mathbf{f}}}{\partial \hat{\theta}} + \underbrace{\left( \frac{\partial g}{\partial \tilde{\mathbf{q}}} + \lambda(t)^\top \frac{\partial \hat{\mathbf{f}}}{\partial \tilde{\mathbf{q}}} + \dot{\lambda}(t)^\top \right)}_{\text{anular}=0} \frac{d\tilde{\mathbf{q}}}{d\hat{\theta}} \right] dt \\
&\quad - \lambda(0)^\top \frac{d\tilde{\mathbf{q}}}{d\hat{\theta}}(0) + \lambda(0)^\top \frac{d\tilde{\mathbf{q}}}{d\hat{\theta}}(0) - \underbrace{\lambda(T)^\top \frac{d\tilde{\mathbf{q}}}{d\hat{\theta}}(T)}_{\text{anular}=0}.
\end{aligned}$$

Reconocemos que el conjunto de términos que multiplican a la sensibilidad  $\frac{d\tilde{\mathbf{q}}}{d\hat{\theta}}$  corresponde a la expresión  $\frac{\partial \mathcal{L}}{\partial \tilde{\mathbf{q}}}$ , la cual anulamos junto con la condición terminal  $\lambda(T) = \mathbf{0}$ . Dado que  $\lambda$  es arbitraria, la ecuación (3.9) se satisface para cualquier valor de la misma. Tras transponer y reordenar los términos, obtenemos las ecuaciones adjuntas deseadas:

$$\dot{\lambda}(t) = - \left[ \left( \frac{\partial \hat{\mathbf{f}}}{\partial \tilde{\mathbf{q}}} \right)^\top \lambda(t) + \left( \frac{\partial g}{\partial \tilde{\mathbf{q}}} \right)^\top \right], \quad \lambda(T) = \mathbf{0}.$$

Por otra parte, tal como demostramos en la configuración del problema (3.10), minimizar el Lagrangiano  $\mathcal{L}$  es equivalente a minimizar el funcional de pérdida  $\tilde{\ell}$ . Asimismo, por definición,  $g$  no depende de  $\hat{\theta}$ , lo que implica que  $\frac{\partial g}{\partial \hat{\theta}} = \mathbf{0}$ . Por consiguiente, la expresión final para el gradiente es:

$$\frac{d\tilde{\ell}(\hat{\theta})}{d\hat{\theta}} = \int_0^T \lambda^\top \frac{\partial \hat{\mathbf{f}}(\tilde{\mathbf{q}}, \hat{\theta})}{\partial \hat{\theta}} dt.$$

Esto completa la demostración. □

**Nota 3.2.2** Para evaluar la ecuación adjunta (3.12) en la práctica, primero se integra el modelo reducido directo (forward model) para obtener  $\tilde{\mathbf{q}}(t, \hat{\theta})$  en el intervalo  $[0, T]$ . Posteriormente, utilizando  $\hat{\mathbf{q}}(t) = \tilde{\mathbf{q}}(t, \hat{\theta})$ , la EDO adjunta (3.12) se resuelve hacia atrás en el tiempo, desde  $t = T$  hasta  $t = 0$ .

### Formulación para la Estructura Cuadrática

Consideramos ahora la estructura cuadrática de OpInf previamente establecida en (2.2) con el fin de derivar las ecuaciones adjuntas específicas para dicho problema. Esta derivación toma inspiración de las notas de [15]. Por conveniencia, denotamos la ecuación de restricción como

$$\mathbf{c}(\hat{\mathbf{q}}(t), \hat{\theta}) := \dot{\hat{\mathbf{q}}}(t) - \hat{\mathbf{c}} - \hat{\mathbf{A}}\hat{\mathbf{q}}(t) - \hat{\mathbf{H}}[\hat{\mathbf{q}}(t) \otimes \hat{\mathbf{q}}(t)] - \hat{\mathbf{B}}\mathbf{u}.$$

Aquí, el operador tensor  $\hat{\mathbf{H}} \in \mathbb{R}^{r \times r^2}$  es Kronecker-simétrico, es decir,  $\hat{\mathbf{H}}[\hat{\mathbf{q}} \otimes \mathbf{v}] = \hat{\mathbf{H}}[\mathbf{v} \otimes \hat{\mathbf{q}}]$ . De este modo, el Lagrangiano (objetivo menos el producto de la variable adjunta por la restricción, menos el término de frontera de la condición inicial) se reescribe como

$$\begin{aligned} \mathcal{L}(\hat{\mathbf{q}}(\cdot), \hat{\theta}, \lambda(\cdot)) &= \ell(\hat{\mathbf{q}}(\cdot)) - \int_0^T \lambda(t)^\top \mathbf{c}(\hat{\mathbf{q}}(t), \hat{\theta}) dt - \lambda(0)^\top (\hat{\mathbf{q}}(0) - \hat{\mathbf{q}}_0) \\ &= \int_0^T g(\hat{\mathbf{q}}(t), t) dt \\ &\quad - \int_0^T \lambda(t)^\top (\dot{\hat{\mathbf{q}}}(t) - \hat{\mathbf{c}} - \hat{\mathbf{A}}\hat{\mathbf{q}}(t) - \hat{\mathbf{H}}(\hat{\mathbf{q}}(t) \otimes \hat{\mathbf{q}}(t)) - \hat{\mathbf{B}}\mathbf{u}(t)) dt \\ &\quad - \lambda(0)^\top (\hat{\mathbf{q}}(0) - \hat{\mathbf{q}}_0), \end{aligned}$$

nde la variable adjunta  $\lambda(\cdot) \in \mathcal{C}^1(\mathcal{T})^r$  posee la misma dimensión que la variable de estado  $\hat{\mathbf{q}}(\cdot)$ . La ecuación adjunta para  $\lambda(\cdot)$  se deduce al imponer la condición de derivada direccional (Gâteaux) nula:

$$\mathbf{D}_{\hat{\mathbf{q}}} \mathcal{L}(\hat{\mathbf{q}}, \hat{\theta}, \lambda; \mathbf{v}) = \frac{d}{d\varepsilon} [\mathcal{L}(\hat{\mathbf{q}} + \varepsilon \mathbf{v}, \lambda, \hat{\theta})]_{\varepsilon=0} = \mathbf{0} \quad \forall \mathbf{v} \in \mathcal{C}^1(\mathcal{T})^r.$$

Observando en primer lugar que

$$\begin{aligned} \frac{d}{d\varepsilon} [\ell(\hat{\mathbf{q}} + \varepsilon \mathbf{v}, \hat{\theta})]_{\varepsilon=0} &= \int_0^T \frac{d}{d\varepsilon} [g(\hat{\mathbf{q}} + \varepsilon \mathbf{v}, t)]_{\varepsilon=0} dt = \int_0^T \nabla_{\hat{\mathbf{q}}} g(\hat{\mathbf{q}}, t)^\top \mathbf{v} dt, \\ \frac{d}{d\varepsilon} [\mathbf{c}(\hat{\mathbf{q}} + \varepsilon \mathbf{v}, \hat{\theta})]_{\varepsilon=0} &= \dot{\mathbf{v}} - \hat{\mathbf{A}}\mathbf{v} - 2\hat{\mathbf{H}}[\hat{\mathbf{q}} \otimes \mathbf{v}], \end{aligned}$$

las ecuaciones adjuntas quedan planteadas de la siguiente forma:

$$\begin{aligned} \mathbf{0} &= \int_0^T \nabla_{\hat{\mathbf{q}}} g(\hat{\mathbf{q}}(t), t)^\top \mathbf{v}(t) dt \\ &\quad - \int_0^T \lambda(t)^\top (\dot{\mathbf{v}}(t) - \hat{\mathbf{A}}\mathbf{v}(t) - 2\hat{\mathbf{H}}[\hat{\mathbf{q}}(t) \otimes \mathbf{v}(t)]) dt - \lambda(0)^\top \mathbf{v}(0). \end{aligned}$$

Buscamos expresar estas ecuaciones adjuntas en su forma débil empleando una fun-

ción de prueba  $\mathbf{v}$ . La integración por partes del término temporal produce

$$\begin{aligned} - \int_0^T \lambda(t)^\top \dot{\mathbf{v}}(t) dt &= - [\lambda(t)^\top \mathbf{v}(t)]_0^T + \int_0^T \dot{\lambda}(t)^\top \mathbf{v}(t) dt \\ &= \lambda(0)^\top \mathbf{v}(0) - \lambda(T)^\top \mathbf{v}(T) + \int_0^T \mathbf{v}(t)^\top \dot{\lambda}(t) dt. \end{aligned}$$

Utilizando esta identidad y transponiendo cada término (escalar), las ecuaciones adjuntas se transforman en

$$\mathbf{0} = \int_0^T \mathbf{v}(t)^\top (\dot{\lambda}(t) + \hat{\mathbf{A}}^\top \lambda(t) + 2\tilde{\mathbf{H}}[\hat{\mathbf{q}}(t) \otimes \lambda(t)] + \nabla_{\hat{\mathbf{q}}} g(\hat{\mathbf{q}}(t), t)) dt - \mathbf{v}(T)^\top \lambda(T),$$

donde  $\tilde{\mathbf{H}} \in \mathbb{R}^{r \times r^2}$  es la matriz que satisface la relación

$$\lambda^\top \tilde{\mathbf{H}}[\hat{\mathbf{q}} \otimes \mathbf{v}] = \mathbf{v}^\top \tilde{\mathbf{H}}[\hat{\mathbf{q}} \otimes \lambda], \quad \forall \hat{\mathbf{q}}, \mathbf{v}, \lambda \in \mathbb{R}^r.$$

Dado que  $\mathbf{v}$  es completamente arbitraria, reconocemos la expresión anterior como la forma débil asociada a la siguiente ecuación en su forma fuerte:

$$-\dot{\lambda}(t) = \hat{\mathbf{A}}^\top \lambda(t) + 2\tilde{\mathbf{H}}[\hat{\mathbf{q}}(t) \otimes \lambda(t)] + \nabla_{\hat{\mathbf{q}}} g(\hat{\mathbf{q}}(t), t), \quad \lambda(T) = \mathbf{0}, \quad t \in [0, T].$$

Este sistema lineal (aunque no autónomo) puede resolverse mediante integración temporal hacia atrás (*backward time-integration*) dada una trayectoria de estados calculada  $\hat{\mathbf{q}}(t)$ . El sistema puede expresarse de una forma lineal más explícita respecto a  $\lambda(t)$  como

$$\begin{aligned} \dot{\lambda}(t) &= -(\hat{\mathbf{A}}^\top + 2\mathbf{M}(\hat{\mathbf{q}}(t))) \lambda(t) - \nabla_{\hat{\mathbf{q}}} g(\hat{\mathbf{q}}(t), t), \\ &= -\nabla_{\hat{\mathbf{q}}} \hat{\mathbf{f}}(\hat{\mathbf{q}}(t), \hat{\theta}) \lambda(t) - \nabla_{\hat{\mathbf{q}}} g(\hat{\mathbf{q}}(t), t), \end{aligned} \quad \lambda(T) = \mathbf{0}, \quad (3.14)$$

donde  $\mathbf{M}(\hat{\mathbf{q}}(t)) \in \mathbb{R}^{r \times r}$  es la matriz definida de modo que

$$\mathbf{M}(\hat{\mathbf{q}}(t)) \lambda = \tilde{\mathbf{H}}[\hat{\mathbf{q}}(t) \otimes \lambda(t)].$$

Postergamos el cómputo detallado de  $\nabla_{\hat{\mathbf{q}}} g$ , junto con los gradientes remanentes requeridos para el método adjunto, para la Sección 3.3.

### Demostración Alternativa (Análisis de Sensibilidad del Problema Primal)

Un enfoque alternativo para demostrar el Teorema 3.2.1, sugerido por la dirección de tesis, proporciona la siguiente deducción. Se incluye en este documento debido a que aporta una perspectiva analítica complementaria muy valiosa sobre el problema.

Partimos de las ecuaciones de sensibilidad *primales* asociadas a la solución del estado  $\tilde{\mathbf{q}}(t, \hat{\theta})$  y a los parámetros  $\hat{\theta}_i$  para  $i = 1, \dots, d$ :

$$\dot{\tilde{\mathbf{q}}}(t) = \hat{\mathbf{f}}(\tilde{\mathbf{q}}(t, \hat{\theta}), \hat{\theta}), \quad \partial_{\hat{\theta}_i} \dot{\tilde{\mathbf{q}}}(t) = \frac{\partial \hat{\mathbf{f}}}{\partial \tilde{\mathbf{q}}}(\tilde{\mathbf{q}}(t, \hat{\theta}), \hat{\theta})^\top \partial_{\hat{\theta}_i} \tilde{\mathbf{q}}(t, \hat{\theta}) + \partial_{\hat{\theta}_i} \hat{\mathbf{f}}(\tilde{\mathbf{q}}(t, \hat{\theta}), \hat{\theta}).$$

Multiplicando la segunda ecuación por una función de prueba arbitraria  $\mathbf{v}(\cdot) \in \mathcal{C}^1(\mathcal{T})^r$ , integrando en el intervalo  $[0, T]$  y reordenando términos, podemos reescribir el problema primal en su forma débil:

$$\int_0^T \left[ \partial_{\hat{\theta}_i} \dot{\tilde{\mathbf{q}}}(t, \hat{\theta}) - \left( \frac{\partial \hat{\mathbf{f}}}{\partial \tilde{\mathbf{q}}} \right)^\top \partial_{\hat{\theta}_i} \tilde{\mathbf{q}}(t, \hat{\theta}) \right]^\top \mathbf{v}(t) dt = \int_0^T (\partial_{\hat{\theta}_i} \hat{\mathbf{f}})^\top \mathbf{v}(t) dt,$$

o bien, utilizando la notación de operadores,

$$\langle \mathcal{L} \partial_{\hat{\theta}_i} \tilde{\mathbf{q}}, \mathbf{v} \rangle = \langle \partial_{\hat{\theta}_i} \hat{\mathbf{f}}, \mathbf{v} \rangle,$$

donde el operador

$$\mathcal{L} : \mathcal{C}^1(\mathcal{T})^r \rightarrow \mathcal{C}^0(\mathcal{T})^r,$$

con dominio y codominio dotados del producto interno habitual de  $L^2$  (denotado por  $\langle \cdot, \cdot \rangle$ ), viene dado por

$$(\mathcal{L} \mathbf{w})(t) = \dot{\mathbf{w}}(t) - \left( \frac{\partial \hat{\mathbf{f}}}{\partial \tilde{\mathbf{q}}} \right)^\top \mathbf{w}(t),$$

para cualquier función  $\mathbf{w}(\cdot) \in \mathcal{C}^1(\mathcal{T})^r$ .

Por otra parte, el gradiente de la función de pérdida  $\tilde{\ell}$  con respecto a la componente paramétrica  $\hat{\theta}_i$  es

$$\frac{d}{d\hat{\theta}_i} \tilde{\ell} = \int_0^T \frac{\partial g}{\partial \tilde{\mathbf{q}}}(\tilde{\mathbf{q}}(t, \hat{\theta}), t)^\top \partial_{\hat{\theta}_i} \tilde{\mathbf{q}}(t, \hat{\theta}) dt = \langle \nabla_{\tilde{\mathbf{q}}} g, \partial_{\hat{\theta}_i} \tilde{\mathbf{q}} \rangle.$$

Ahora, definimos el problema adjunto introduciendo una variable adjunta  $\lambda(\cdot) \in \mathcal{C}^1(\mathcal{T})^r$  tal que

$$\langle \mathbf{w}, \tilde{\mathcal{L}} \lambda \rangle = \langle \nabla_{\tilde{\mathbf{q}}} g, \mathbf{w} \rangle \quad \forall \mathbf{w}(\cdot) \in \mathcal{C}^1(\mathcal{T})^r \text{ con } \mathbf{w}(0) = \mathbf{0},$$

donde  $\tilde{\mathcal{L}}$  es el operador adjunto de  $\mathcal{L}$ . Por la definición de operador adjunto, esto es equivalente a escribir

$$\langle \mathcal{L} \mathbf{w}, \boldsymbol{\lambda} \rangle = \langle \nabla_{\tilde{\mathbf{q}}} g, \mathbf{w} \rangle.$$

Seleccionando de forma específica  $\mathbf{w} = \partial_{\hat{\theta}_i} \tilde{\mathbf{q}}$  en esta identidad, se obtiene de inmediato de forma directa:

$$\frac{d}{d\hat{\theta}_i} \tilde{\ell} = \langle \mathcal{L} \partial_{\hat{\theta}_i} \tilde{\mathbf{q}}, \boldsymbol{\lambda} \rangle \underset{\substack{\text{problema primal} \\ \text{con } \mathbf{v} = \boldsymbol{\lambda}}}{=} \langle \partial_{\hat{\theta}_i} \hat{\mathbf{f}}, \boldsymbol{\lambda} \rangle = \int_0^T \boldsymbol{\lambda}^\top \partial_{\hat{\theta}_i} \hat{\mathbf{f}}(t, \hat{\boldsymbol{\theta}}) dt.$$

Para deducir la EDO adjunta en sí, expandimos la relación estructural:

$$\langle \mathcal{L} \mathbf{w}, \boldsymbol{\lambda} \rangle = \int_0^T \left[ \dot{\mathbf{w}}(t) - \left( \frac{\partial \hat{\mathbf{f}}}{\partial \tilde{\mathbf{q}}} \right)^\top \mathbf{w}(t) \right]^\top \boldsymbol{\lambda}(t) dt = \int_0^T \left( \frac{\partial g}{\partial \tilde{\mathbf{q}}} \right)^\top \mathbf{w}(t) dt = \langle \nabla_{\tilde{\mathbf{q}}} g, \mathbf{w} \rangle.$$

Integrando por partes el término de la derivada temporal

$$\int_0^T \dot{\mathbf{w}}(t)^\top \boldsymbol{\lambda}(t) dt = [\mathbf{w}(t)^\top \boldsymbol{\lambda}(t)]_0^T - \int_0^T \dot{\boldsymbol{\lambda}}(t)^\top \cdot \mathbf{w}(t) dt,$$

utilizando la condición de contorno  $\mathbf{w}(0) = \mathbf{0}$  e imponiendo la condición terminal natural  $\boldsymbol{\lambda}(T) = \mathbf{0}$  para anular el término de frontera en el límite superior, obtenemos

$$\int_0^T [-\dot{\boldsymbol{\lambda}}(t)]^\top \mathbf{w}(t) dt = \int_0^T \left[ \left( \frac{\partial \hat{\mathbf{f}}}{\partial \tilde{\mathbf{q}}} \right)^\top \boldsymbol{\lambda}(t) + \left( \frac{\partial g}{\partial \tilde{\mathbf{q}}} \right)^\top \right]^\top \mathbf{w}(t) dt.$$

Puesto que la función  $\mathbf{w}$  es arbitraria, los integrandos deben coincidir puntualmente, proporcionando la expresión final para las ecuaciones del estado adjunto:

$$\dot{\boldsymbol{\lambda}}(t) = - \left[ \left( \frac{\partial \hat{\mathbf{f}}}{\partial \tilde{\mathbf{q}}} \right)^\top \boldsymbol{\lambda}(t) + \left( \frac{\partial g}{\partial \tilde{\mathbf{q}}} \right)^\top \right], \quad \boldsymbol{\lambda}(T) = \mathbf{0}.$$

□

**Nota 3.2.3** Mientras que las ecuaciones adjuntas resultan independientes del índice  $i = 1, \dots, d$  (es decir, de la dimensión del vector de parámetros  $\hat{\boldsymbol{\theta}}$ ), las ecuaciones de sensibilidad primales exigen resolver  $d$  sistemas acoplados independientes para el conjunto  $\{\partial_{\hat{\theta}_i} \tilde{\mathbf{q}}(t)\}_{i=1}^d$ . Esto otorga al método adjunto una ventaja computacional masiva frente al método primal cuando el espacio de parámetros es de alta dimensión ( $d \gg 1$ ).

## Validación Teórica y Empírica

Una cuestión crucial que surge en este punto es determinar si la solución provista por el método adjunto  $\boldsymbol{\lambda}(t)$ , obtenida mediante la resolución de la EDO de propagación hacia atrás, es matemáticamente exacta y fiable para guiar los procesos de optimización numérica de  $\hat{\boldsymbol{\theta}}$ . Para abordar esto, exponemos argumentos de validación tanto teóricos como empíricos que respaldan el uso del enfoque adjunto.

La exactitud (salvo por el error numérico de truncamiento) del gradiente de la función de pérdida calculado a través del método adjunto se sustenta en tres pilares fundamentales de la teoría de optimización matemática:

- **Condiciones de Optimalidad:** Las ecuaciones adjuntas emergen de manera directa de las condiciones de Karush-Kuhn-Tucker (KKT) para optimización con restricciones continuas [16]. Al formular el Lagrangiano tal como se muestra en (3.8), las condiciones de estacionariedad de primer orden respecto al estado  $\hat{\mathbf{q}}$  dictan de forma unívoca la EDO adjunta, garantizando que los gradientes obtenidos respeten la estructura de las restricciones dinámicas.
- **Dualidad:** La variable adjunta  $\lambda(t)$  posee una interpretación analítica fundamental como medida de sensibilidad. En concreto, cuantifica cómo se propagan las perturbaciones infinitesimales introducidas en el vector de estado  $\hat{\mathbf{q}}(t)$  a lo largo del horizonte temporal y cómo afectan al valor final del funcional de pérdida. Esta perspectiva de dualidad es matemáticamente equivalente al modo inverso de diferenciación automática (*reverse-mode automatic differentiation*) empleado en el ámbito del aprendizaje profundo [5], donde los gradientes fluyen hacia atrás a través de grafos computacionales.
- **Consistencia con el Principio del Máximo de Pontryagin:** Definiendo el Hamiltoniano asociado al problema de control óptimo como

$$\mathcal{H}(\hat{\mathbf{q}}, \lambda, \hat{\theta}) = \lambda^\top \hat{\mathbf{f}}(\hat{\mathbf{q}}, \hat{\theta}) + g(\hat{\mathbf{q}}, t),$$

el principio de Pontryagin prescribe la ecuación del coestado mediante

$$\dot{\lambda}(t) = - \left( \frac{\partial \mathcal{H}}{\partial \hat{\mathbf{q}}} \right)^\top = - \left( \frac{\partial \hat{\mathbf{f}}}{\partial \hat{\mathbf{q}}} \right)^\top \lambda(t) - \left( \frac{\partial g}{\partial \hat{\mathbf{q}}} \right)^\top.$$

El método adjunto recupera exactamente estas condiciones necesarias de optimalidad tipificadas en la teoría clásica de control óptimo [17].

En conclusión, el método adjunto calcula el coestado o variable adjunta  $\lambda(t)$  mediante la imposición rigurosa de las condiciones de optimalidad de primer orden del Lagrangiano en un entorno dinámico continuo. Su sólido fundamento en el marco teórico de Karush-Kuhn-Tucker, su interpretación desde la teoría de dualidad y su perfecta consistencia con el principio de Pontryagin aseguran que los gradientes resultantes sean exactos dentro de la precisión de la máquina. Complementado con estrategias de validación empírica en fase de código (como el chequeo de gradientes mediante diferencias finitas, el monitoreo estricto de las curvas de convergencia y la evaluación de la plausibilidad física del modelo reducido), el método adjunto constituye un mecanismo de alta precisión y matemáticamente robusto para la optimización de parámetros en sistemas dinámicos.

### 3.3. Ecuaciones de los Tres Gradientes

Obsérvese que el método adjunto utilizado en este trabajo requiere el cálculo de los tres gradientes siguientes:

$$\frac{\partial \hat{\mathbf{f}}}{\partial \hat{\mathbf{q}}}, \quad \frac{\partial g}{\partial \hat{\mathbf{q}}}, \quad \text{y} \quad \frac{\partial \hat{\mathbf{f}}}{\partial \hat{\theta}},$$

donde

$$\hat{\mathbf{f}} : \mathbb{R}^r \times \mathbb{R}^d \rightarrow \mathbb{R}^r$$

denota el lado derecho de la dinámica del modelo de orden reducido (ROM-OpInf) en (2.10), y

$$g : \mathbb{R}^r \times [0, T] \rightarrow \mathbb{R}$$

define el integrando del funcional de pérdida  $\ell$  en (3.1). El cálculo eficiente de estos gradientes es crucial tanto para la integración hacia atrás de la ecuación adjunta como para la evaluación de la derivada total de la pérdida reducida  $\tilde{\ell}$ . A continuación, se presenta una derivación detallada de las expresiones analíticas de cada uno de estos gradientes.

#### 3.3.1. $\nabla_{\hat{\mathbf{q}}} \hat{\mathbf{f}}(\hat{\mathbf{q}}, \hat{\theta})$

Comenzamos recordando la definición del operador dinámico:

$$\hat{\mathbf{f}}(\hat{\mathbf{q}}, \hat{\theta}) = \hat{\mathbf{c}} + \hat{\mathbf{A}}\hat{\mathbf{q}} + \hat{\mathbf{H}}(\hat{\mathbf{q}} \otimes \hat{\mathbf{q}}) + \hat{\mathbf{B}}\mathbf{u}.$$

Para calcular la derivada de  $\hat{\mathbf{f}}$  respecto a  $\hat{\mathbf{q}}$ , consideramos la derivada de Gâteaux en la dirección de un vector arbitrario  $\mathbf{v} \in \mathbb{R}^r$ :

$$\mathbf{D}_{\hat{\mathbf{q}}} \hat{\mathbf{f}}(\hat{\mathbf{q}}, \hat{\theta}; \mathbf{v}) = \mathbf{D}_{\hat{\mathbf{q}}} \hat{\mathbf{f}}[\mathbf{v}] = \lim_{\varepsilon \rightarrow 0} \frac{\hat{\mathbf{f}}(\hat{\mathbf{q}} + \varepsilon \mathbf{v}, \hat{\theta}) - \hat{\mathbf{f}}(\hat{\mathbf{q}}, \hat{\theta})}{\varepsilon} = \frac{d}{d\varepsilon} [\hat{\mathbf{f}}(\hat{\mathbf{q}} + \varepsilon \mathbf{v}, \hat{\theta})]_{\varepsilon=0}.$$

Desarrollando la expresión, obtenemos:

$$\hat{\mathbf{f}}(\hat{\mathbf{q}} + \varepsilon \mathbf{v}, \hat{\theta}) = \hat{\mathbf{c}} + \hat{\mathbf{A}}(\hat{\mathbf{q}} + \varepsilon \mathbf{v}) + \hat{\mathbf{H}}((\hat{\mathbf{q}} + \varepsilon \mathbf{v}) \otimes (\hat{\mathbf{q}} + \varepsilon \mathbf{v})) + \hat{\mathbf{B}}\mathbf{u}.$$

El término cuadrático se expande de la siguiente forma:

$$(\hat{\mathbf{q}} + \varepsilon \mathbf{v}) \otimes (\hat{\mathbf{q}} + \varepsilon \mathbf{v}) = \hat{\mathbf{q}} \otimes \hat{\mathbf{q}} + \varepsilon(\hat{\mathbf{q}} \otimes \mathbf{v} + \mathbf{v} \otimes \hat{\mathbf{q}}) + \varepsilon^2(\mathbf{v} \otimes \mathbf{v}).$$

Sustituyendo este desarrollo en la expresión de  $\hat{\mathbf{f}}$ , se deduce que:

$$\hat{\mathbf{f}}(\hat{\mathbf{q}} + \varepsilon \mathbf{v}, \hat{\theta}) = \hat{\mathbf{c}} + \hat{\mathbf{A}}\hat{\mathbf{q}} + \varepsilon \hat{\mathbf{A}}\mathbf{v} + \hat{\mathbf{H}}(\hat{\mathbf{q}} \otimes \hat{\mathbf{q}}) + \varepsilon \hat{\mathbf{H}}(\hat{\mathbf{q}} \otimes \mathbf{v} + \mathbf{v} \otimes \hat{\mathbf{q}}) + \hat{\mathbf{B}}\mathbf{u} + \mathcal{O}(\varepsilon^2).$$

Diferenciando con respecto a  $\varepsilon$  y evaluando en  $\varepsilon = 0$ , resulta:

$$\mathbf{D}_{\hat{\mathbf{q}}} \hat{\mathbf{f}}(\hat{\mathbf{q}}, \hat{\theta}; \mathbf{v}) = \hat{\mathbf{A}}\mathbf{v} + 2\hat{\mathbf{H}}(\hat{\mathbf{q}} \otimes \mathbf{v}),$$

donde se ha utilizado el hecho de que  $\hat{\mathbf{H}}$  es Kronecker-simétrico, es decir,  $\hat{\mathbf{H}}(\hat{\mathbf{q}} \otimes \mathbf{v}) = \hat{\mathbf{H}}(\mathbf{v} \otimes \hat{\mathbf{q}})$ . Igualando las derivadas direccionales con sus respectivos productos internos, se obtiene:

$$(\hat{\mathbf{A}} + 2\hat{\mathbf{H}}(\hat{\mathbf{q}} \otimes \mathbf{I}_r))\mathbf{v} = \mathbf{D}_{\hat{\mathbf{q}}} \hat{\mathbf{f}}(\hat{\mathbf{q}}, \hat{\theta}; \mathbf{v}) = (\nabla_{\hat{\mathbf{q}}} \hat{\mathbf{f}})^\top \mathbf{v},$$

donde  $\mathbf{I}_r$  denota la matriz identidad de tamaño  $r$ . Por lo tanto, el gradiente viene dado por:

$$\nabla_{\hat{\mathbf{q}}} \hat{\mathbf{f}}(\hat{\mathbf{q}}, \hat{\theta}) = \hat{\mathbf{A}}^\top + (2\hat{\mathbf{H}}(\hat{\mathbf{q}} \otimes \mathbf{I}_r))^\top = \hat{\mathbf{A}}^\top + 2\mathbf{M}(\hat{\mathbf{q}}) \in \mathbb{R}^{r \times r}, \quad (3.15)$$

donde definimos la matriz asimétrica  $\mathbf{M}(\hat{\mathbf{q}}) := (\hat{\mathbf{H}}(\hat{\mathbf{q}} \otimes \mathbf{I}_r))^\top \in \mathbb{R}^{r \times r}$ .

Cabe destacar que este resultado es plenamente consistente con la formulación analítica presentada en el primer ejemplo de demostración relativo a la estructura cuadrática en (3.14).

### 3.3.2. $\nabla_{\hat{\mathbf{q}}} g(\hat{\mathbf{q}}, t)$

Por definición, el integrando de la función de pérdida viene dado por

$$g(\hat{\mathbf{q}}(t), t) = \|\hat{\mathbf{q}}(t) - \hat{\mathbf{q}}_{\text{true}}(t)\|_2^2,$$

donde  $\hat{\mathbf{q}}(t)$  representa el vector de estado predicho al tiempo  $t$ , y  $\hat{\mathbf{q}}_{\text{true}}(t)$  es el vector de estado correspondiente a los datos reales en ese mismo instante. Introducimos, por conveniencia analítica, el vector residual:

$$\mathbf{r}(\hat{\mathbf{q}}(t), t) := \hat{\mathbf{q}}(t) - \hat{\mathbf{q}}_{\text{true}}(t).$$

De este modo, la derivada de Gâteaux de  $g$  en la dirección  $\mathbf{v} \in \mathbb{R}^r$  se expresa como

$$\begin{aligned} \mathbf{D}_{\hat{\mathbf{q}}} g[\mathbf{v}] &= \frac{d}{d\varepsilon} g(\hat{\mathbf{q}}(t) + \varepsilon \mathbf{v}, t) \Big|_{\varepsilon=0} = \frac{d}{d\varepsilon} \left( \|\hat{\mathbf{q}}(t) + \varepsilon \mathbf{v} - \hat{\mathbf{q}}_{\text{true}}(t)\|_2^2 \right) \Big|_{\varepsilon=0} \\ &= \frac{d}{d\varepsilon} \left( \|\mathbf{r}(\hat{\mathbf{q}}(t), t) + \varepsilon \mathbf{v}\|_2^2 \right) \Big|_{\varepsilon=0} \\ &= 2(\mathbf{r}(\hat{\mathbf{q}}(t), t) + \varepsilon \mathbf{v})^\top \mathbf{v} \Big|_{\varepsilon=0} \\ &= 2(\mathbf{r}(\hat{\mathbf{q}}(t), t))^\top \mathbf{v}, \end{aligned}$$

donde la igualdad de la derivada de la norma se deduce de la identidad diferencial:

$$\frac{d}{d\varepsilon} \|\mathbf{r} + \varepsilon \mathbf{v}\|^2 = \frac{d}{d\varepsilon} [(\mathbf{r} + \varepsilon \mathbf{v})^\top (\mathbf{r} + \varepsilon \mathbf{v})] = 2(\mathbf{r} + \varepsilon \mathbf{v})^\top \mathbf{v}.$$

Por consiguiente, identificamos directamente la expresión para el gradiente final:

$$\nabla_{\hat{\mathbf{q}}(t)} g(\hat{\mathbf{q}}(t), t) = 2(\hat{\mathbf{q}}(t) - \hat{\mathbf{q}}_{\text{true}}(t)) \in \mathbb{R}^r. \quad (3.16)$$

### 3.3.3. $\nabla_{\hat{\theta}} \hat{\mathbf{f}}(\hat{\mathbf{q}}, \hat{\theta})$

Recordemos que, de acuerdo con la Definición 3.1.1, el vector global de parámetros se define estructurado como

$$\hat{\theta} = \left[ \underbrace{\hat{\mathbf{c}}}_{\in \mathbb{R}^r}, \underbrace{\text{vec}(\hat{\mathbf{A}})}_{\in \mathbb{R}^{r^2}}, \underbrace{\text{vec}(\hat{\mathbf{H}})}_{\in \mathbb{R}^{r^3}}, \underbrace{\text{vec}(\hat{\mathbf{B}})}_{\in \mathbb{R}^{rm}} \right] \in \mathbb{R}^d, \quad d = r + r^2 + r^3 + rm.$$

Con el fin de calcular cada bloque de matrices de manera independiente, dividimos el gradiente en cuatro subbloques estructurales:

$$\nabla_{\hat{\theta}} \hat{\mathbf{f}} = \begin{bmatrix} \nabla_{\hat{\mathbf{c}}} \hat{\mathbf{f}} \\ \nabla_{\hat{\mathbf{A}}} \hat{\mathbf{f}} \\ \nabla_{\hat{\mathbf{H}}} \hat{\mathbf{f}} \\ \nabla_{\hat{\mathbf{B}}} \hat{\mathbf{f}} \end{bmatrix} :$$

#### (i) Gradiente respecto a $\hat{\mathbf{c}}$ .

Para cualquier dirección  $\mathbf{v}_{\hat{\mathbf{c}}} \in \mathbb{R}^r$ ,

$$\mathbf{D}_{\hat{\mathbf{c}}} \hat{\mathbf{f}}(\hat{\mathbf{q}}, \hat{\theta}; \mathbf{v}_{\hat{\mathbf{c}}}) = \left. \frac{d}{d\varepsilon} (\hat{\mathbf{c}} + \varepsilon \mathbf{v}_{\hat{\mathbf{c}}}) \right|_{\varepsilon=0} = \mathbf{v}_{\hat{\mathbf{c}}} = \mathbf{I}_r \mathbf{v}_{\hat{\mathbf{c}}} = (\nabla_{\hat{\mathbf{c}}} \hat{\mathbf{f}})^\top \mathbf{v}_{\hat{\mathbf{c}}},$$

por lo tanto,

$$\nabla_{\hat{\mathbf{c}}} \hat{\mathbf{f}} = \mathbf{I}_r \in \mathbb{R}^{r \times r}.$$

#### (ii) Gradiente respecto a $\hat{\mathbf{A}}$ .

Para cualquier matriz  $\mathbf{V}_A \in \mathbb{R}^{r \times r}$  con  $\hat{\mathbf{v}}_A = \text{vec}(\mathbf{V}_A) \in \mathbb{R}^{r^2}$ ,

$$\mathbf{D}_{\hat{\mathbf{A}}} \hat{\mathbf{f}}(\hat{\mathbf{q}}, \hat{\theta}; \mathbf{V}_A) = \left. \frac{d}{d\varepsilon} ((\hat{\mathbf{A}} + \varepsilon \mathbf{V}_A) \hat{\mathbf{q}}) \right|_{\varepsilon=0} = \mathbf{V}_A \hat{\mathbf{q}} = (\hat{\mathbf{q}} \otimes \mathbf{I}_r)^\top \hat{\mathbf{v}}_A = (\nabla_{\hat{\mathbf{A}}} \hat{\mathbf{f}})^\top \hat{\mathbf{v}}_A,$$

de donde se deduce que

$$\nabla_{\hat{\mathbf{A}}} \hat{\mathbf{f}} = \hat{\mathbf{q}} \otimes \mathbf{I}_r \in \mathbb{R}^{r^2 \times r}.$$

#### (iii) Gradiente respecto a $\hat{\mathbf{H}}$ .

Para cualquier configuración  $\mathbf{V}_H \in \mathbb{R}^{r \times r^2}$  con  $\tilde{\mathbf{v}}_H = \text{vec}(\mathbf{V}_H) \in \mathbb{R}^{r^3}$ ,

$$\mathbf{D}_{\hat{\mathbf{H}}} \hat{\mathbf{f}}(\hat{\mathbf{q}}, \hat{\theta}; \mathbf{V}_H) = \left. \frac{d}{d\varepsilon} \left( (\hat{\mathbf{H}} + \varepsilon \mathbf{V}_H) (\hat{\mathbf{q}} \otimes \hat{\mathbf{q}}) \right) \right|_{\varepsilon=0} = \mathbf{V}_H (\hat{\mathbf{q}} \otimes \hat{\mathbf{q}}) = ((\hat{\mathbf{q}} \otimes \hat{\mathbf{q}}) \otimes \mathbf{I}_r)^\top \tilde{\mathbf{v}}_H = (\nabla_{\hat{\mathbf{H}}} \hat{\mathbf{f}})^\top \tilde{\mathbf{v}}_H,$$

lo que produce

$$\nabla_{\hat{\mathbf{H}}} \hat{\mathbf{f}} = (\hat{\mathbf{q}} \otimes \hat{\mathbf{q}}) \otimes \mathbf{I}_r \in \mathbb{R}^{r^3 \times r}.$$

(iv) Gradiente respecto a  $\hat{\mathbf{B}}$ .

De manera análoga, se demuestra que

$$\nabla_{\hat{\mathbf{B}}} \hat{\mathbf{f}} = \mathbf{u} \otimes \mathbf{I}_r \in \mathbb{R}^{rm \times r}.$$

Por conveniencia para la posterior implementación algorítmica, expresamos los gradientes respecto a  $\hat{\theta}$  en su forma transpuesta, acoplándose así con la formulación del método adjunto del funcional de pérdida descrita en (3.13). De este modo, reuniendo los cuatro bloques transpuestos, la expresión final para el operador del gradiente resulta en

$$\nabla_{\hat{\theta}}^\top \hat{\mathbf{f}}(\hat{\mathbf{q}}, \hat{\theta}) = [\mathbf{I}_r, \hat{\mathbf{q}}^\top \otimes \mathbf{I}_r, (\hat{\mathbf{q}} \otimes \hat{\mathbf{q}})^\top \otimes \mathbf{I}_r, \mathbf{u}^\top \otimes \mathbf{I}_r] \in \mathbb{R}^{r \times d}. \quad (3.17)$$

### 3.4. Descenso de Gradiente para la Optimización de Parámetros

Una vez obtenido el gradiente del funcional de pérdida reducido  $\tilde{\ell}(\hat{\theta})$  mediante el método adjunto, procedemos a minimizar  $\tilde{\ell}$  sobre el espacio paramétrico  $\hat{\theta} \in \mathbb{R}^d$  utilizando el algoritmo de descenso de gradiente (GD) [18, 19]. Concretamente, partiendo de un vector de parámetros inicial  $\hat{\theta}^0$ , generamos una secuencia  $\{\hat{\theta}^j\}_{j \geq 0}$  mediante la actualización iterativa de los parámetros en la dirección del gradiente negativo (dirección de máximo descenso):

$$\hat{\theta}^{j+1} = \hat{\theta}^j - \eta_j \nabla \tilde{\ell}(\hat{\theta}^j), \quad j = 0, 1, 2, \dots \quad (3.18)$$

donde

- $\nabla \tilde{\ell}(\hat{\theta}^j)$  representa el gradiente exacto evaluado mediante las ecuaciones adjuntas,
- $\eta_j > 0$  es el tamaño de paso (tasa de aprendizaje) en la iteración  $j$ .

El procedimiento se detiene una vez que se satisface un criterio de parada convergente. Una condición de parada habitual viene dada por

$$\|\nabla \tilde{\ell}(\hat{\theta}^j)\|_2 \leq \epsilon, \quad (3.19)$$

para una tolerancia prescrita  $\epsilon > 0$ . Bajo una selección adecuada de la secuencia  $\{\eta_j\}$  y asumiendo hipótesis suaves sobre  $\tilde{\ell}$ , las iteraciones  $\hat{\theta}^j$  convergen hacia un punto estacionario del sistema.

### Selección de la Tasa de Aprendizaje y Consideraciones de Convergencia

La elección de  $\eta_j$  afecta directamente el comportamiento de convergencia. El uso de un tamaño de paso constante  $\eta_j = \eta$  está muy extendido, pero requiere un ajuste minucioso para equilibrar la velocidad de convergencia y la estabilidad numérica.

Introducimos a continuación dos definiciones fundamentales que se emplearán en el análisis numérico de convergencia del método de descenso de gradiente [20].

**Definición 3.4.1 (Función  $L$ -suave)** Se dice que una función diferenciable  $\mathcal{J} : \mathbb{R}^p \rightarrow \mathbb{R}$  es  $L$ -suave si existe una constante Lipschitz  $L > 0$  tal que, para todo par de vectores  $\omega, \tilde{\omega} \in \mathbb{R}^p$ ,

$$\|\nabla \mathcal{J}(\omega) - \nabla \mathcal{J}(\tilde{\omega})\|_2 \leq L \|\omega - \tilde{\omega}\|_2. \quad (3.20)$$

La ecuación (3.20) establece que el operador gradiente de  $\mathcal{J}$  es Lipschitz continuo con constante  $L$ .

**Definición 3.4.2 (Función  $\mu$ -fuertemente convexa)** Se dice que una función  $\mathcal{J} : \mathbb{R}^p \rightarrow \mathbb{R}$  es  $\mu$ -fuertemente convexa si existe una constante de curvatura  $\mu > 0$  tal que, para todo par de vectores  $\omega, \tilde{\omega} \in \mathbb{R}^p$ ,

$$\mathcal{J}(\tilde{\omega}) \geq \mathcal{J}(\omega) + \nabla \mathcal{J}(\omega)^\top (\tilde{\omega} - \omega) + \frac{\mu}{2} \|\tilde{\omega} - \omega\|_2^2. \quad (3.21)$$

Esta condición implica que la función  $\mathcal{J}$  posee una cota inferior estricta en su curvatura, garantizando convexidad global. Una caracterización equivalente de la  $\mu$ -convexidad fuerte se puede formular en términos de la matriz Hessiana:

$$\mathcal{J} \text{ es } \mu\text{-fuertemente convexa} \quad \Leftrightarrow \quad \nabla^2 \mathcal{J}(\omega) \geq \mu \mathbf{I}, \quad \forall \omega \in \mathbb{R}^p \quad (3.22)$$

donde  $\nabla^2 \mathcal{J}(\omega)$  denota el Hessiano de  $\mathcal{J}$  evaluado en  $\omega$ , e  $\mu \mathbf{I}$  representa la matriz identidad escalada por el factor  $\mu$ . Esto asegura que todos los autovalores del Hessiano son como mínimo  $\mu$ , afianzando una cota inferior de convexidad.

Bajo las hipótesis de suavidad  $L$  y convexidad fuerte  $\mu$  de la función objetivo  $\mathcal{J}(\omega)$ , junto con un tamaño de paso acotado, el método de GD converge de forma garantizada hacia un mínimo global único de  $\mathcal{J}(\omega)$  [20]. La Tabla 3.1 resume las tasas de error de

convergencia de GD bajo distintas condiciones de suavidad y convexidad asociadas a diferentes elecciones del tamaño de paso.

| Condición                                                                                                                                                         | Tamaño de Paso ( $\eta$ ) | Tasa de Error                                                                                     |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------|---------------------------------------------------------------------------------------------------|
| $\mathcal{J}$ es $L$ -suave                                                                                                                                       | $\frac{1}{L}$             | $\mathcal{J}(\omega^j) - \mathcal{J}(\omega^*) \leq \frac{1}{j\eta} \ \omega^0 - \omega^*\ _2^2$  |
| $\mathcal{J}$ es $L$ -suave y $\mu$ -fuertemente convexa                                                                                                          | $\frac{1}{L}$             | $\ \omega^j - \omega^*\ _2 \leq \left(1 - \frac{\mu}{L}\right)^j \ \omega^0 - \omega^*\ _2$       |
| $\mathcal{J}$ es $L$ -suave y $\mu$ -fuertemente convexa                                                                                                          | $\frac{2}{L + \mu}$       | $\ \omega^j - \omega^*\ _2 \leq \left(\frac{L - \mu}{L + \mu}\right)^j \ \omega^0 - \omega^*\ _2$ |
| Nota: $\frac{L - \mu}{L + \mu} = \frac{\kappa - 1}{\kappa + 1}$ , donde $\kappa := \frac{L}{\mu}$ es el número de condición de el Hessiano $\nabla^2 \mathcal{J}$ |                           |                                                                                                   |

**Tabla 3.1:** Tasas de error de convergencia de GD bajo diferentes hipótesis de suavidad y convexidad.

Para funciones  $\mu$ -fuertemente convexas  $\mathcal{J}$ , destacan dos propiedades esenciales que aseguran un comportamiento de optimización altamente eficiente:

- Unicidad del mínimo absoluto, asegurando una solución del problema de optimización bien planteada.
- Convergencia rápida, garantizando una tasa de convergencia al menos lineal con un decaimiento exponencial del error.

En el marco de la Inferencia de Operadores (Operator Inference), frecuentemente se añade un término de regularización  $L^2$  a la función de pérdida. El objetivo primordial de esto no es forzar la convexidad fuerte para el análisis de convergencia basado en gradiente, sino mejorar el condicionamiento numérico y estabilizar algebraicamente la solución del sistema [9]. Esta regularización modifica la pérdida por mínimos cuadrados penalizando la norma de Frobenius del operador inferido, conduciendo al siguiente funcional objetivo regularizado:

$$\|\mathbf{D}\hat{\mathbf{O}}^T - \mathbf{Z}^T\|_F^2 + \xi^2 \|\hat{\mathbf{O}}^T\|_F^2,$$

donde  $\mathbf{D} \in \mathbb{R}^{k \times d}$  representa la matriz de datos,  $\hat{\mathbf{O}} \in \mathbb{R}^{r \times d}$  es la matriz de operadores (parámetros),  $\mathbf{Z} \in \mathbb{R}^{r \times k}$  denota la matriz de derivadas temporales (datos del lado izquierdo) y  $\xi > 0$  representa el hiperparámetro de regularización. El término regularizador promueve el buen planteamiento del problema asegurando que la matriz de ecuaciones normales pase a ser estrictamente definida positiva, lo cual mitiga los problemas mal condicionados y previene el sobreajuste (*overfitting*) ante datos de entrenamiento ruidosos. Si bien esto optimiza la estabilidad numérica de la solución, no garantiza de manera directa tasas de convergencia rápidas para GD, especialmente cuando el valor de  $\xi$  es pequeño y la función objetivo carece de convexidad fuerte global.

### Tamaño de Paso Adaptativo: Búsqueda de Línea de Armijo

En lugar de fijar un valor estático para  $\eta_j$ , el Algoritmo 1 implementa un procedimiento de retroceso (*backtracking*) para seleccionar adaptativamente el valor de  $\eta_j$  de modo que se satisfaga la condición de descenso suficiente de Armijo [16]:

---

**Algorithm 1:** Búsqueda de Línea por Retroceso de Armijo + Descenso de Gradiente

---

**Paso 1. Inicialización:** Seleccionar  $\alpha \in (0, 1)$ ,  $\beta \in (0, 1)$  y un tamaño de paso inicial  $\eta_0 > 0$ . Fijar el contador de iteraciones  $j = 0$ ;

**Paso 2. Calcular la dirección de descenso:** Evaluar el gradiente exacto  $\mathcal{G}^j = \nabla \tilde{\ell}(\hat{\theta}^j)$ ;

**Paso 3. Bucle de retroceso (*backtracking*):;**

$\eta \leftarrow \eta_0$ ;

**while**  $\tilde{\ell}(\hat{\theta}^j - \eta \mathcal{G}^j) > \tilde{\ell}(\hat{\theta}^j) - \alpha \eta \|\mathcal{G}^j\|_2^2$  **do**

$\eta \leftarrow \beta \eta$ ;

**end**

**Paso 4. Actualización:** Fijar el tamaño de paso óptimo  $\eta_j = \eta$  y actualizar el vector de parámetros mediante;

$\hat{\theta}^{j+1} = \hat{\theta}^j - \eta_j \mathcal{G}^j$ ;

**Paso 5. Criterio de parada:** Si  $\|\mathcal{G}^{j+1}\|_2 \leq \epsilon$  (u otro criterio de convergencia) finalizar; en caso contrario hacer  $j \leftarrow j + 1$  y volver al Paso 2.;

---

Para entornos estocásticos o problemas a gran escala, suelen preferirse variantes como el Descenso de Gradiente Estocástico (SGD) [18] debido a sus menores requerimientos de memoria y coste computacional por iteración. No obstante, para conjuntos de datos de dimensiones moderadas (como en los experimentos numéricos detallados posteriormente), se favorece el uso del Descenso de Gradiente clásico gracias a su mayor precisión matemática. A pesar de que el método de GD acoplado con búsqueda de línea de Armijo aplicado a problemas no convexos (como el diseño de optimización restringida que abordamos) solo asegura convergencia hacia algún punto estacionario, en nuestra metodología realizamos un arranque en caliente (*warm-start*) del algoritmo de GD. Para ello, tomamos como solución inicial los operadores obtenidos mediante la regresión algebraica de Operator Inference en (2.12). Dado que esta aproximación inicial se sitúa ya en la vecindad de un mínimo de alta calidad, el proceso de GD converge con gran rapidez y decrece drásticamente la probabilidad de quedar atrapado en puntos estacionarios subóptimos.

### 3.5. Algoritmo del Método Adjunto

A continuación, se detalla el algoritmo completo de entrenamiento basado en el estado adjunto, el cual integra las etapas de preprocesamiento de datos, modelado de orden reducido y refinamiento de parámetros mediante optimización por gradiente. El Algoritmo 2 sintetiza el flujo de trabajo derivado en las secciones previas: tras recopilar y preprocesar los datos del modelo de alta fidelidad, se calcula una estimación paramétrica inicial mediante Operator Inference [9], para luego resolver iterativamente los sistemas directo y adjunto, computar los gradientes exactos y ejecutar el descenso de gradiente con retroceso hasta alcanzar la convergencia de los operadores.

---

**Algorithm 2:** Método Adjunto para el Entrenamiento de Parámetros
 

---

**Input:** Instantáneas del modelo completo  $\mathbf{Q}, \mathbf{U}$ ; parámetros de retroceso  $(\alpha, \beta, \eta_0)$ ; tolerancia de convergencia  $\epsilon$ ; intervalo temporal de entrenamiento  $[t_1, t_k]$ ; número máximo de iteraciones  $j_{max}$ .

**Output:** Parámetros optimizados del modelo reducido  $\hat{\theta}^*$ .

**Paso 1. Recopilación y preprocesamiento de datos:** Reunir las matrices de instantáneas  $\mathbf{Q} = [\mathbf{q}_{t_1}, \dots, \mathbf{q}_{t_k}]$ ,  $\mathbf{U} = [\mathbf{u}_{t_1}, \dots, \mathbf{u}_{t_k}]$ . [Opcional] Ejecutar transformaciones de escalado, centrado o transformaciones no lineales (*lifting*);

**Paso 2. Reducción de la dimensionalidad:** Calcular la base ortonormal POD  $\mathbf{V}_r$  (mediante criterio de energía acumulada o fijando una dimensión de ROM constante  $r$ ). Proyectar los estados del sistema:  $\hat{\mathbf{Q}} \approx \mathbf{V}_r^T \mathbf{Q}$ ;

**Paso 3. Estimación paramétrica inicial:** Resolver el problema de regresión lineal de OpInf utilizando  $\hat{\mathbf{Q}}$  para obtener los operadores iniciales  $\hat{\theta}_{\text{OpInf}}^*$ ;

**Paso 4. Bucle de descenso de gradiente:** Inicializar el contador  $j = 0$ , fijar los parámetros iniciales  $\hat{\theta}^0 = \hat{\theta}_{\text{OpInf}}^*$  y definir los límites del tamaño de paso. Ejecutar iterativamente;;

**4.1. Resolución del modelo directo (*forward*):** Calcular la trayectoria reducida  $\tilde{\mathbf{q}}(t, \hat{\theta}^j)$  resolviendo la EDO de orden reducido;

**4.2. Cálculo del residual de estados:** Evaluar el vector residual continuo  $\mathbf{r}(\tilde{\mathbf{q}}, t) := \tilde{\mathbf{q}}(t, \hat{\theta}^j) - \hat{\mathbf{q}}_{\text{true}}(t)$ ;

**4.3. Resolución del modelo adjunto:** Integrar la EDO del estado adjunto (3.12) hacia atrás en el tiempo para determinar  $\lambda(t)$ , aplicando las relaciones dadas en (3.15) y (3.16);

**4.4. Ensamblaje del gradiente:** Construir el vector del gradiente global  $\nabla \tilde{\ell}(\hat{\theta}^j)$  según la ecuación (3.13) empleando la estructura de bloques deducida en (3.17);

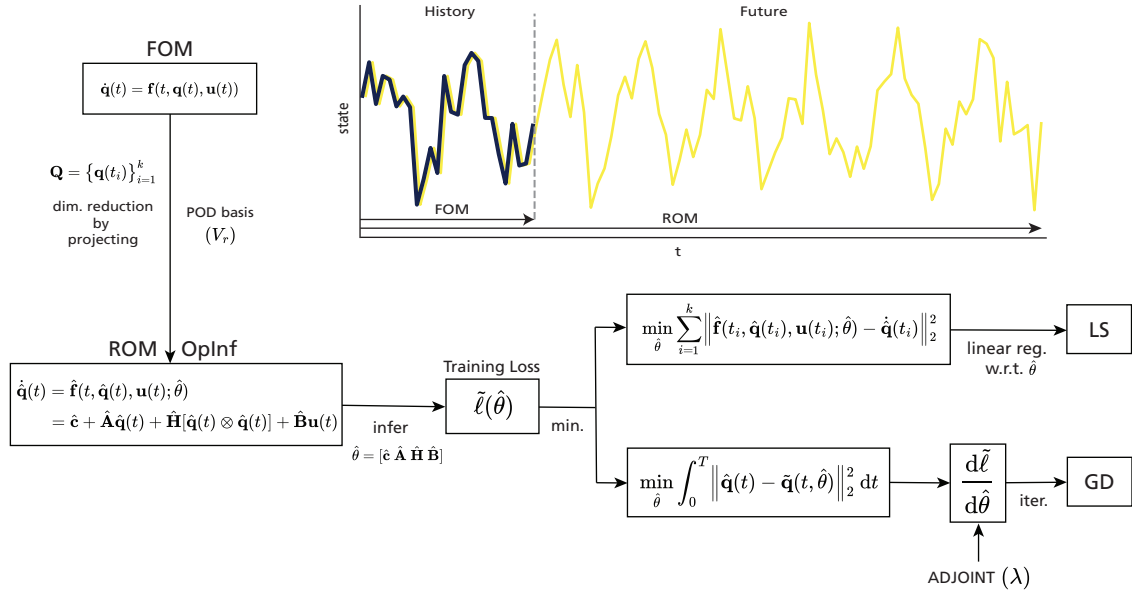
**4.5. Búsqueda de línea de Armijo:** Determinar el tamaño de paso adaptativo  $\eta_j$  mediante retroceso empleando el esquema del Algoritmo 1;

**4.6. Actualización paramétrica:** Actualizar los operadores del modelo reducido mediante:  $\hat{\theta}^{j+1} = \hat{\theta}^j - \eta_j \nabla \tilde{\ell}(\hat{\theta}^j)$ ;

**4.7. Evaluación de criterios de parada:** Verificar la convergencia del sistema. Si  $\|\nabla \tilde{\ell}(\hat{\theta}^{j+1})\|_2 \leq \epsilon$  o bien  $j = j_{max}$  finalizar el bucle; en caso contrario hacer  $j \leftarrow j + 1$  y repetir desde el paso 4.1;

**Paso 5. Retornar** el vector final de operadores optimizados  $\hat{\theta}^* = \hat{\theta}^{j+1}$ ;

---



**Figura 3.1:** Esquema del proceso de inferencia de modelos de orden reducido (ROM); flujo algebraico clásico de OpInf (rama superior) y refinamiento basado en el método adjunto dinámico (rama inferior).

## EXPERIMENTOS NUMÉRICOS

En este capítulo, evaluamos el método de entrenamiento propuesto basado en el estado adjunto en dos EDP no lineales canónicas: la ecuación de Burgers viscosa 1D y la ecuación de Fisher-KPP 2D. Estos casos de prueba se seleccionaron para desafiar el marco de modelización de orden reducido de formas distintas: la ecuación de Burgers con un parámetro de viscosidad pequeño  $\nu = 0,01$  produce gradientes pronunciados e interacciones no lineales de advección-difusión, mientras que la ecuación de Fisher-KPP exhibe una dinámica de reacción-difusión en dos dimensiones espaciales.

Para cada modelo, comparamos el método adjunto con el enfoque estándar de OpInf [9] bajo dos escenarios de perturbación:

- (I) reduciendo la densidad de muestreo temporal de las instantáneas (\*snapshots\*) de entrenamiento, y
- (II) añadiendo ruido sintético a los datos.

Planteamos la hipótesis de que, dado que el método adjunto calcula gradientes exactos del funcional de pérdida, proporcionará estimaciones paramétricas más robustas y precisas que OpInf (el cual depende de aproximaciones por diferencias finitas de las derivadas temporales), especialmente cuando los datos de entrenamiento son ruidosos o están muestreados de forma dispersa.

Todos los experimentos comparten una configuración numérica común. Las soluciones estándar de OpInf se obtienen a través del paquete de Python `opinf` proporcionado por el Willcox Research Group [9]. Construimos una base POD  $\mathbf{V}_r$  que captura al menos el 99,9 % de la energía de las instantáneas ( $\kappa_r \geq 0,999$ ). En la regresión OpInf, empleamos esquemas de diferencias finitas de hasta segundo y sexto orden ('ord2', 'ord6') para aproximar  $\dot{\mathbf{Q}}$ . Las integrales temporales se evalúan mediante la regla del trapecio acumulada de `scipy.integrate` [21], y las ecuaciones adjuntas se resuelven hacia atrás en el tiempo con un esquema de Euler implícito. Los parámetros de la búsqueda de línea por retroceso de Armijo se fijan en  $\alpha = 10^{-4}$ ,  $\beta = 0,5$ ,  $\eta_0 = 10^{-3}$ , y el

descenso de gradiente finaliza al alcanzar un valor de tolerancia de  $\epsilon = 10^{-8}$  o tras 1000 iteraciones (véase la implementación del Algoritmo 2 en el Apéndice B.3). Por último, utilizamos los parámetros optimizados  $\hat{\theta}^*$  para predecir nuevas trayectorias resolviendo el ROM aprendido mediante el solver `'solve_ivp(method="BDF")'` de SciPy [22].

## 4.1. Ecuación de Burgers Viscosa

### Generación de Datos (Snapshots)

Para generar el conjunto de datos para la reducción de modelo mediante Inferencia de Operadores, resolvemos la ecuación de Burgers viscosa unidimensional (para la implementación en Python, véase el Apéndice B.1):

$$q_t + q q_x = \nu q_{xx}, \quad x \in [0, 1], \quad t \in [0, T], \quad \nu = 0.01,$$

sujeta a condiciones de contorno de Dirichlet homogéneas

$$q(t, 0) = q(t, 1) = 0,$$

y a la condición inicial de onda senoidal

$$q(x, 0) = \sin(2\pi x).$$

A continuación, se utiliza el siguiente esquema de discretización por diferencias finitas:

- Malla temporal: tiempo final  $T = 1$ , número de pasos  $M = 9999$ , por lo que  $\Delta t = T/M$ .
- Malla espacial: longitud del dominio  $L = 1$ ,  $N = 2^7$  nodos interiores,  $\Delta x = L/(N + 1)$ ; enumeramos los puntos de la malla como  $x_j = j \Delta x$  para  $j = 0, \dots, N + 1$ .
- Término de convección (Lax-Wendroff): denotemos por  $\mathbf{w}^m = \mathbf{LW}(\mathbf{q}^m) \in \mathbb{R}^{N+2}$  la actualización de Lax-Wendroff del estado  $\mathbf{q}^m$ . Su  $j$ -ésima componente viene dada por

$$w_j^m = q_j^m - \frac{\Delta t}{2\Delta x} q_j^m (q_{j+1}^m - q_{j-1}^m) + \frac{\Delta t^2}{2\Delta x^2} q_j^m \left[ \frac{1}{2}(q_{j+1}^m - q_{j-1}^m)^2 + q_j^m (q_{j+1}^m - 2q_j^m + q_{j-1}^m) \right].$$

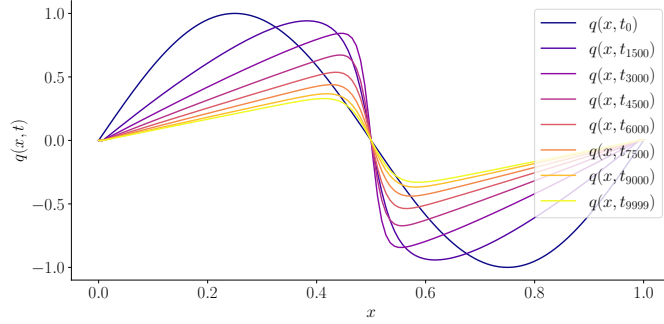
Las condiciones de contorno de Dirichlet se imponen fijando  $w_0^m = w_{N+1}^m = 0$ .

- Término de difusión (Regla del trapecio):

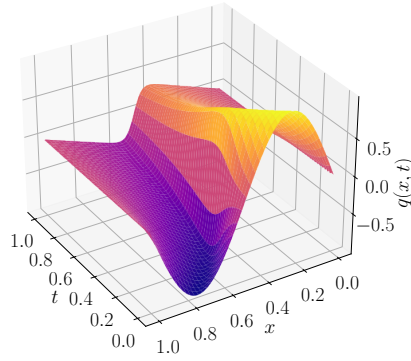
$$\left( \mathbf{I} - \frac{\nu \Delta t}{2} \mathbf{T}_{\Delta x} \right) \mathbf{q}^{m+1} = \mathbf{LW}(\mathbf{q}^m) + \frac{\nu \Delta t}{2} \mathbf{T}_{\Delta x} \mathbf{q}^m,$$

donde  $\mathbf{T}_{\Delta x} = \text{tridiag}\{1, -2, 1\}$  es la matriz estándar de segundas diferencias con condiciones de contorno de Dirichlet cero.

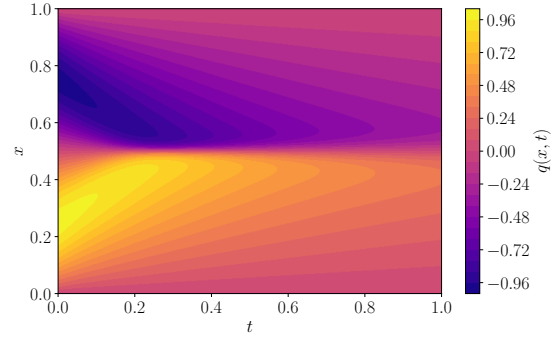
Al apilar la solución de este sistema lineal disperso de ecuaciones para cada instante de tiempo se obtiene la matriz de instantáneas  $\mathbf{Q}_{\text{FOM}} = \mathbf{Q} \in \mathbb{R}^{(N+2) \times (M+1)}$ , que utilizamos para la reducción de modelo. En la Figura 4.1 se muestran tres vistas de esta dinámica.



(a) Perfiles  $q(x, t_k)$  en tiempos seleccionados  $t_k$ .



(b) Superficie 3D de  $q(x, t)$  para  $\nu = 0,01$ .



(c) Contorno espacio-temporal de  $q(x, t)$ .

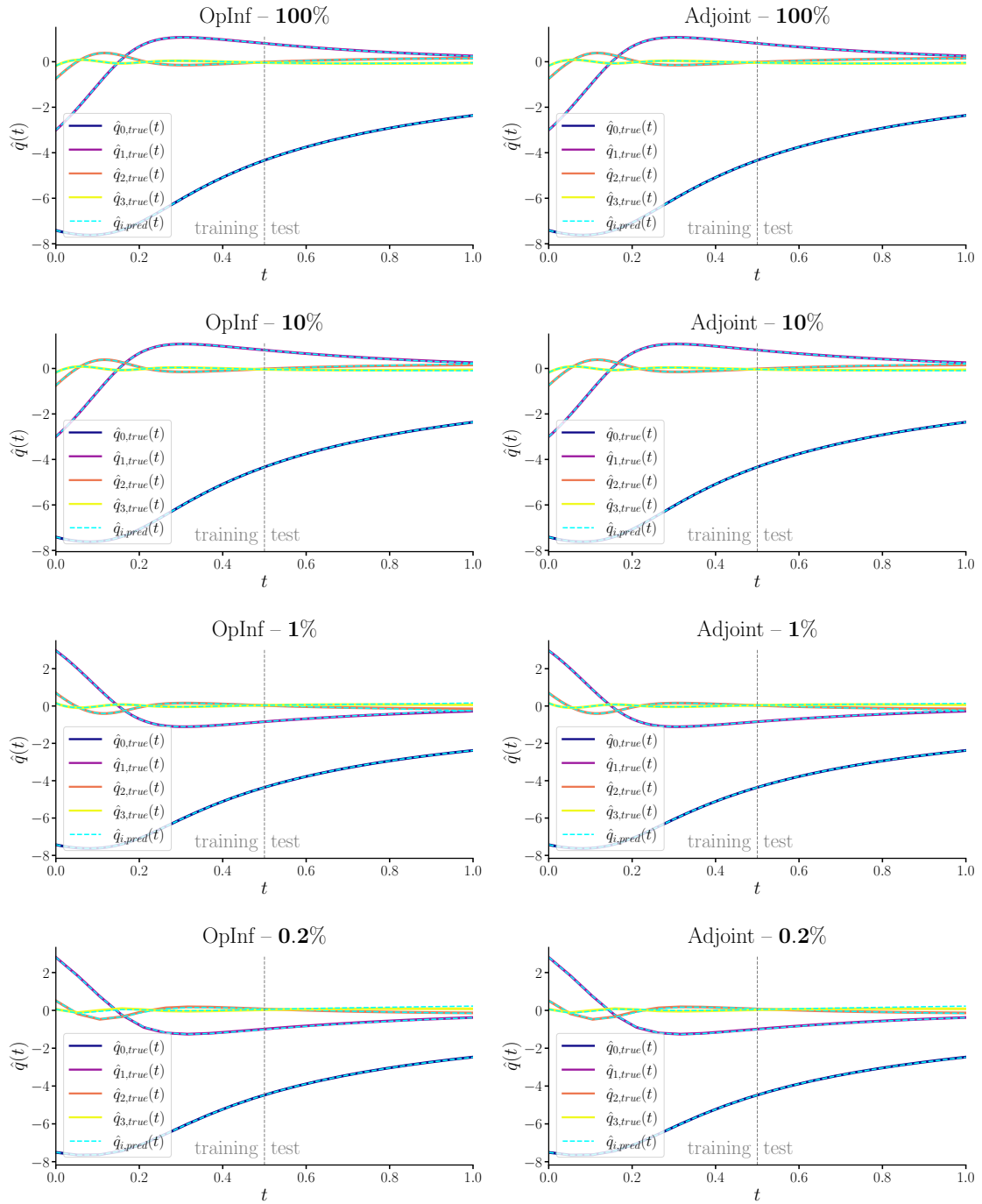
**Figura 4.1:** Dinámica de la ecuación de Burgers viscosa utilizada para la generación de datos.

A partir de la matriz de datos de instantáneas de Burgers  $\mathbf{Q}$ , aplicamos en primer lugar el Algoritmo 2 para obtener el vector de parámetros óptimos  $\hat{\theta}^*$ , utilizando como datos de entrenamiento las instantáneas del intervalo de tiempo  $t \in [0, 0,5]$ . A continuación, empleamos estos parámetros aprendidos para predecir la evolución del sistema en el intervalo restante  $t \in (0,5, 1,0]$  mediante la integración del modelo dinámico reducido con el solver implícito BDF `solve_ivp` de SciPy [22]. En el primer conjunto de pruebas, reducimos sistemáticamente el número de instantáneas al 100 % (10 000 puntos), 10 % (1 000 puntos), 1 % (100 puntos) y 0.2 % (20 puntos) de la malla temporal original para simular datos dispersos. En cada escenario, estimamos los parámetros de OpInf tanto con diferencias finitas de segundo orden ('ord2') como de sexto orden ('ord6'), aplicando un peso de regularización  $L^2$  de  $\xi = 10^{-2}$ .

La Figura 4.2 muestra, para cada nivel de muestreo, los estados reducidos reales  $\hat{\mathbf{q}}_{\text{true}}$  (línea continua) frente a las trayectorias predichas  $\hat{\mathbf{q}}_{\text{pred}}$  (línea discontinua) tanto para el modelo OpInf como para el entrenamiento mediante el método adjunto. Aquí, cada curva  $\hat{q}_{i,\text{true}}(t)$  corresponde a la  $i$ -ésima fila (hasta la dimensión  $r$ ) de la matriz de datos

de instantáneas reducida  $\hat{\mathbf{Q}}$ , y  $\hat{q}_{i,\text{pred}}(t)$  denota su correspondiente valor de trayectoria predicho en el tiempo  $t$ . Incluso bajo una dispersión de muestreo extrema, el método adjunto produce predicciones que son visualmente indistinguibles de las de OpInf, demostrando su resiliencia en ausencia de ruido.

## Reducción de la Densidad de Datos

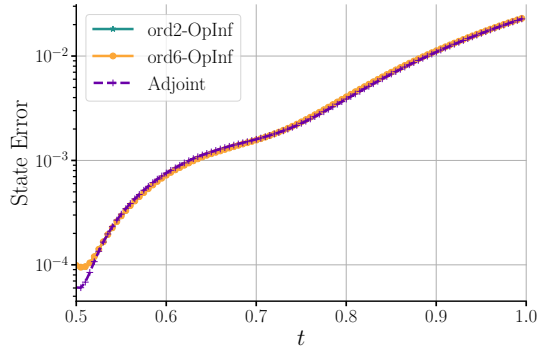


**Figura 4.2:** Resultados de la reducción de la densidad de datos para el experimento sintético de la ecuación de Burgers.

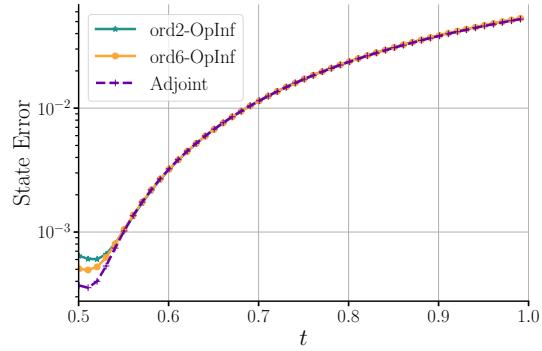
Para cuantificar el impacto de la dispersión de los datos de entrenamiento, calculamos el error de predicción en la región de prueba  $t \in [0.5, 1]$  para cada densidad de muestreo. Específicamente, resolvemos el problema de valor inicial del ROM-OpInf con los operadores aprendidos y comparamos las trayectorias frente a los estados reducidos reales para cada nivel de muestreo (100 %, 10 %, 1 %, 0.2 % respectivamente):

$$\|\hat{\mathbf{q}}_{\text{true}}(t) - \hat{\mathbf{q}}_{\text{pred}}(t)\|_2, \quad t \in [0.5, 1].$$

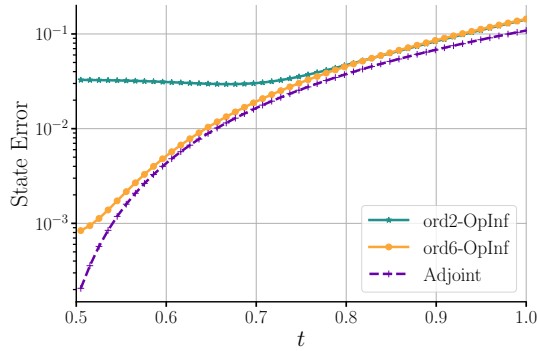
La Figura 4.3 compara las curvas de error de OpInf y del método adjunto a lo largo de todos los niveles de muestreo. El método adjunto produce errores similares o incluso inferiores a los obtenidos cuando se utiliza el esquema de sexto orden, confirmando su precisión al capturar la información de la derivada temporal bajo datos limitados.



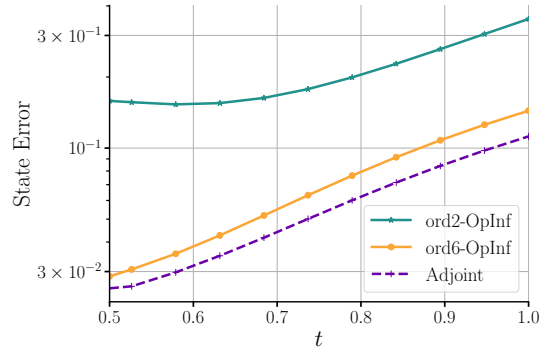
(a) 100 % de los datos.



(b) 10 % de los datos.



(c) 1 % de los datos.



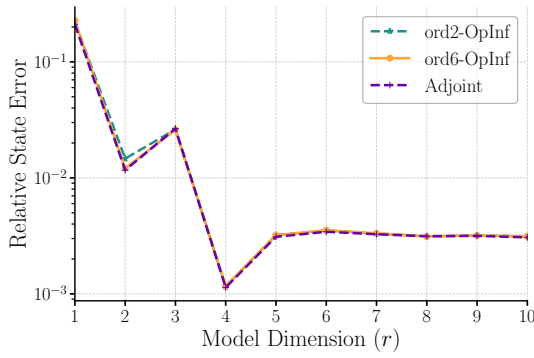
(d) 0.2 % de los datos.

**Figura 4.3:** Error de predicción frente al tiempo para cada ejecución de reducción de densidad de datos en el experimento de la ecuación de Burgers.

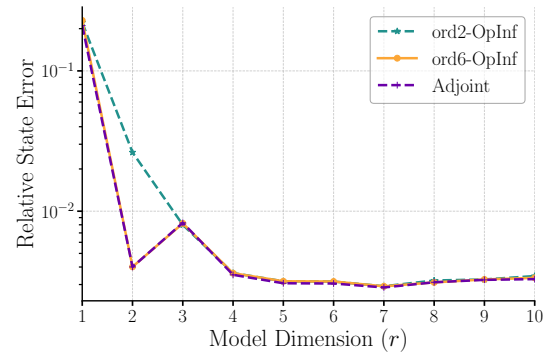
La Figura 4.4 muestra el error relativo

$$\frac{\|\hat{\mathbf{q}}_{\text{true}}(t) - \hat{\mathbf{q}}_{\text{pred}}(t)\|_2}{\|\hat{\mathbf{q}}_{\text{true}}(t)\|_2}, \quad t \in [0, 1],$$

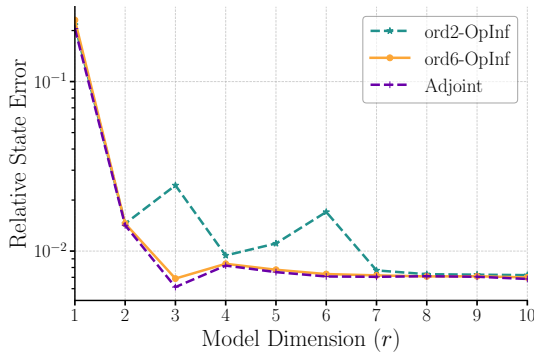
como función de la dimensión del ROM  $r$ . De nuevo, cada subfigura corresponde a una de las cuatro densidades de muestreo. Ambos métodos exhiben el decaimiento esperado en el error relativo conforme aumenta  $r$ . Sin embargo, en el caso de dispersión más extrema (0.2 %), el modelo entrenado mediante el método adjunto alcanza errores marginalmente menores, lo que sugiere que puede construir bases reducidas más eficientes cuando la información de las instantáneas es escasa.



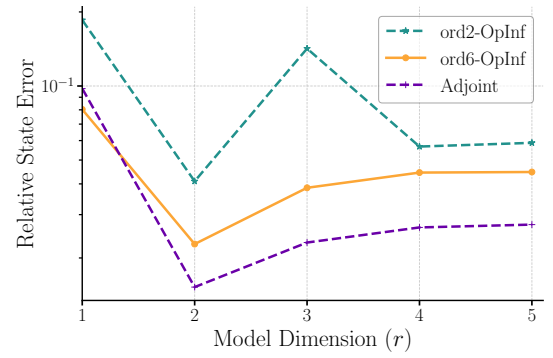
(a) 100 % de los datos.



(b) 10 % de los datos.



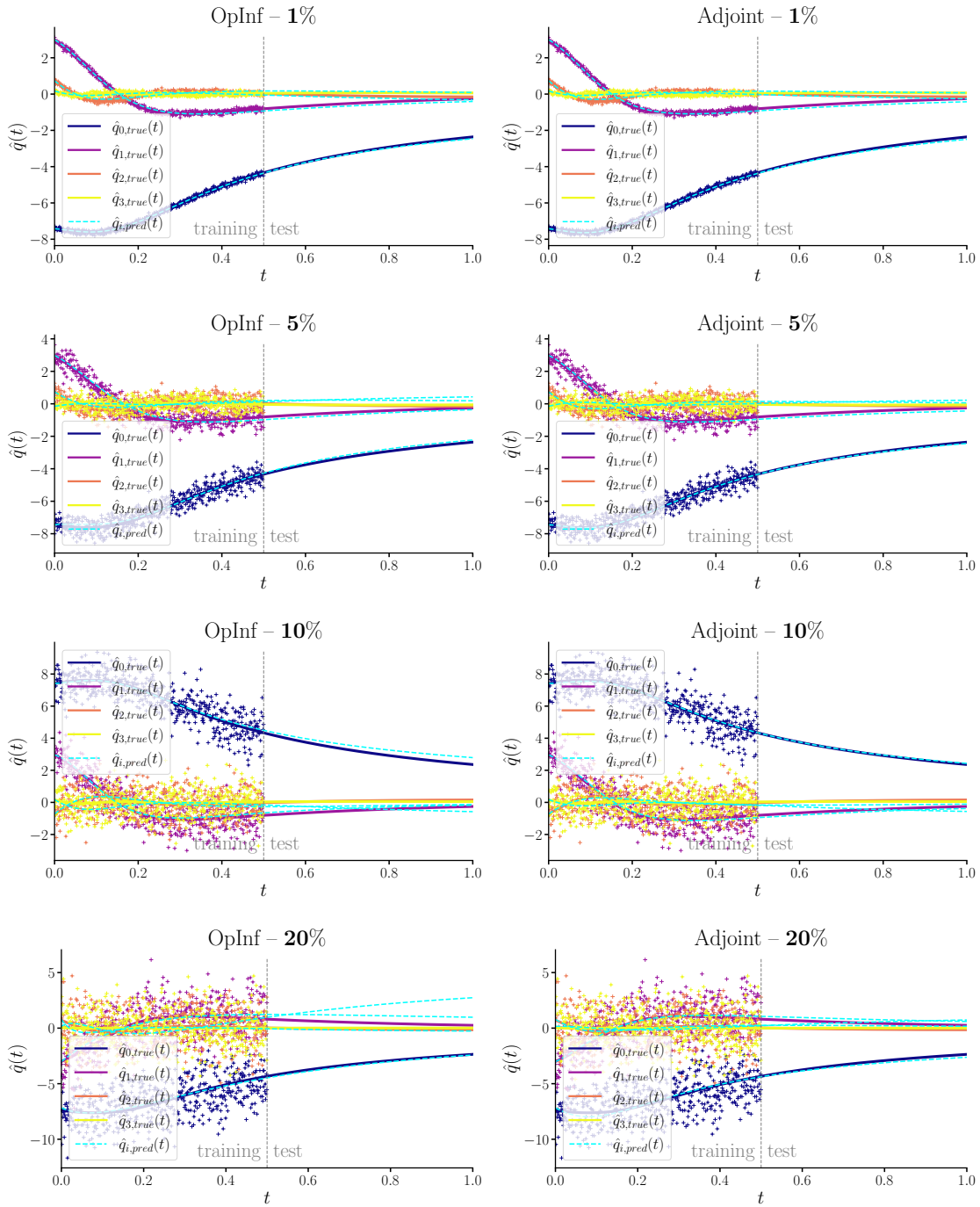
(c) 1 % de los datos.



(d) 0.2 % de los datos.

**Figura 4.4:** Error relativo frente a  $r$  para cada ejecución de reducción de densidad de datos en el experimento de la ecuación de Burgers.

### Perturbación por Ruido



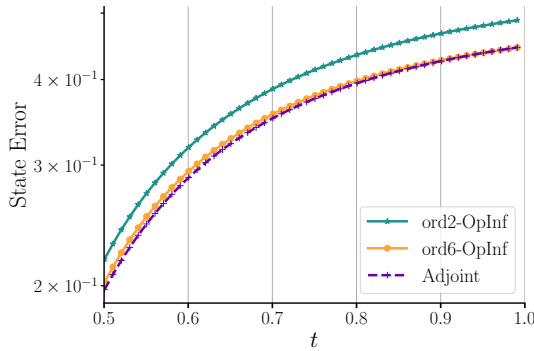
**Figura 4.5:** Simulaciones de perturbación por ruido para el experimento sintético de la ecuación de Burgers.

Para el segundo conjunto de pruebas, con el fin de evaluar la robustez bajo datos imperfectos, corrompemos las instantáneas del modelo completo con ruido gaussiano aditivo. Específicamente, extraemos

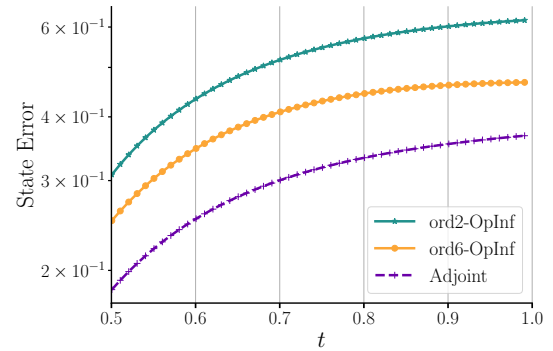
$$\text{error} \sim \mathcal{N}(0, \sigma^2), \quad \sigma = \frac{\text{pct}}{100} \max_{1 \leq j \leq k} \|\mathbf{q}(t_j)\|_2,$$

para niveles de ruido del 1 %, 5 %, 10 % y 20 %. Cada conjunto de datos ruidosos se utiliza para entrenar tanto el modelo OpInf como el adjunto sobre  $t \in [0, 0.5]$ , realizando las predicciones en  $[0.5, 1]$ . La Figura 4.5 visualiza los estados reducidos reales (puntos y línea continua) frente a las predicciones entrenadas (línea discontinua) para ambos métodos en cada nivel de ruido. A medida que aumenta el ruido, el ROM entrenado mediante el método adjunto sigue las trayectorias reales de manera consistentemente más precisa que OpInf, destacando la fidelidad superior de su gradiente en presencia de corrupción de datos.

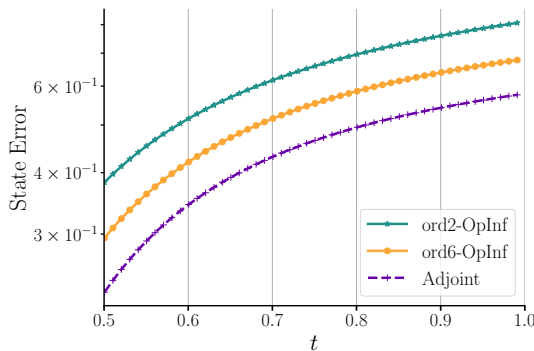
La Figura 4.6 muestra el error de predicción dependiente del tiempo, tal como se definió previamente, para cada nivel de ruido. Aunque todos los métodos se degradan a medida que el ruido crece, el método adjunto mantiene errores más bajos a lo largo del tiempo que ambos enfoques de OpInf.



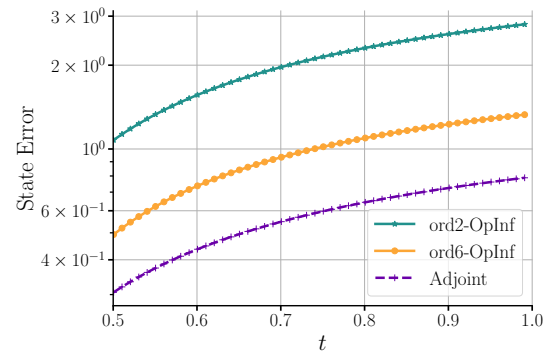
(a) 1 % de nivel de ruido.



(b) 5 % de nivel de ruido.



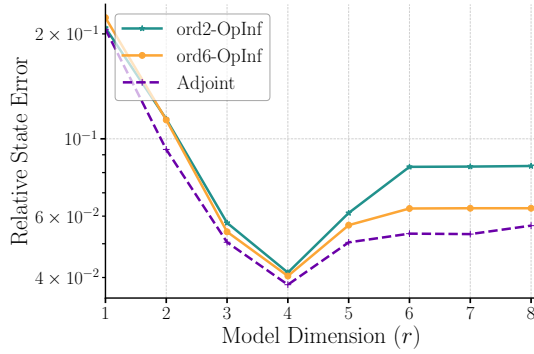
(c) 10 % de nivel de ruido.



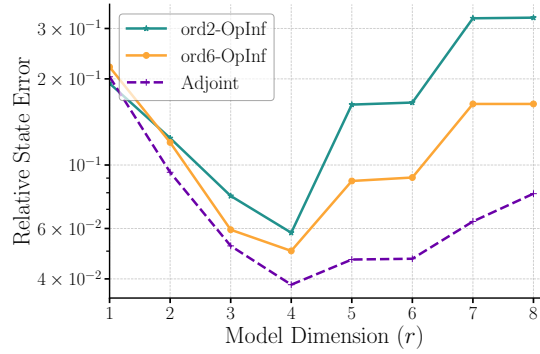
(d) 20 % de nivel de ruido.

**Figura 4.6:** Error de predicción frente al tiempo para cada ejecución de nivel de ruido en el experimento de la ecuación de Burgers.

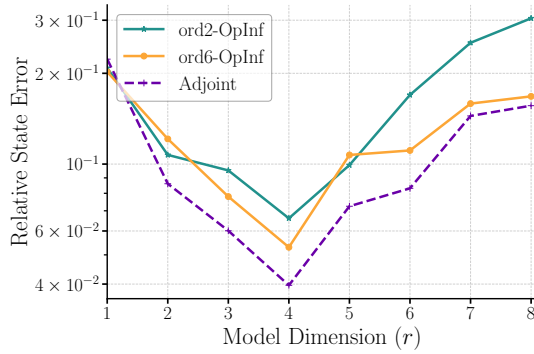
Para cuantificar aún más la eficiencia del modelo, la Figura 4.7 representa el error relativo en función de la dimensión del modelo  $r$ . Para cada nivel de ruido, observamos que el ROM entrenado mediante el método adjunto logra una mejor precisión para el mismo número de modos que ambos métodos OpInf, especialmente con un 5 %, 10 % y 20 % de ruido. Esto confirma que el gradiente adjunto proporciona actualizaciones de parámetros más informativas incluso cuando los datos de entrenamiento están corrompidos.



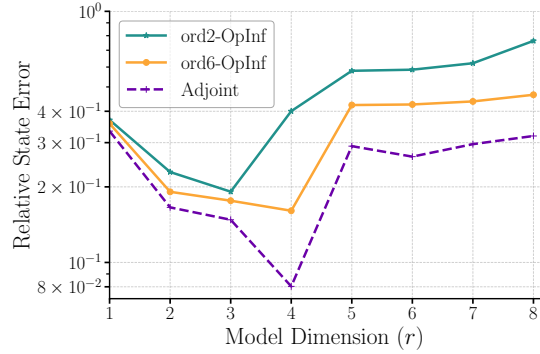
(a) 1 % de nivel de ruido.



(b) 5 % de nivel de ruido.



(c) 10 % de nivel de ruido.



(d) 20 % de nivel de ruido.

**Figura 4.7:** Error relativo frente a  $r$  para cada ejecución de perturbación por ruido en el experimento de la ecuación de Burgers.

## 4.2. Ecuación de Fisher-KPP

### Generación de Datos de Instantáneas

Para este segundo experimento sintético, generamos los datos de entrenamiento (véase el código en el Apéndice B.2) resolviendo numéricamente la ecuación de Fisher-KPP bidimensional,

$$q_t = \mathfrak{D}(q_{xx} + q_{yy}) + \rho q(1 - q), \quad (x, y) \in [0, L_x] \times [0, L_y], \quad t \in [0, T],$$

con condiciones de contorno de Neumann homogéneas (flujo nulo)

$$\left. \frac{\partial q}{\partial n} \right|_{\partial\Omega} = 0,$$

y una condición inicial gaussiana centrada en el dominio:

$$q(x, y, 0) = \exp\left(-10\left[(x - L_x/2)^2 + (y - L_y/2)^2\right]\right).$$

Se consideran la malla y los parámetros siguientes:

- Tamaños del dominio:  $L_x = L_y = 10$ .
- Malla:  $N_x = N_y = 100$  puntos en cada dirección,  $\Delta x = L_x/(N_x - 1)$ ,  $\Delta y = L_y/(N_y - 1)$ .
- Pasos de tiempo: tiempo final  $T = 5$ ,  $\Delta t = 0,005$ ,  $N_t = T/\Delta t = 1000$ .
- Parámetros del modelo: coeficiente de difusión  $\mathfrak{D} = 0,1$ , tasa de crecimiento logístico  $\rho = 1,0$ .

A continuación, se aplica un método de diferencias finitas para las discretizaciones espaciales:

- Para la dimensión  $x$ , construimos una matriz de segundas diferencias 1D  $\mathbf{L}_x^{1D} \in \mathbb{R}^{N_x \times N_x}$  con condiciones de contorno de Neumann mediante

$$(\mathbf{L}_x^{1D})_{jj} = -\frac{2}{\Delta x^2}, \quad (\mathbf{L}_x^{1D})_{j,j\pm 1} = \frac{1}{\Delta x^2}, \quad (\mathbf{L}_x^{1D})_{0,1} = (\mathbf{L}_x^{1D})_{N_x-1,N_x-2} = \frac{2}{\Delta x^2}.$$

- De manera similar, definimos  $\mathbf{L}_y^{1D} \in \mathbb{R}^{N_y \times N_y}$  sobre la malla  $y$ .
- Ambas expresiones se combinan para formar un Laplaciano 2D completo mediante sumas de Kronecker

$$\mathbf{L} = \mathbf{I}_{N_y} \otimes \mathbf{L}_x^{1D} + \mathbf{L}_y^{1D} \otimes \mathbf{I}_{N_x} \in \mathbb{R}^{(N_x N_y) \times (N_x N_y)}.$$

Para el esquema de pasos temporales, utilizamos un paso de Crank-Nicolson semiimplícito para la difusión y un Euler explícito para el término de reacción:

$$\frac{\mathbf{q}^{m+1} - \mathbf{q}^m}{\Delta t} = \frac{\mathfrak{D}}{2} \mathbf{L}(\mathbf{q}^{m+1} + \mathbf{q}^m) + \rho \mathbf{q}^m \circ (\mathbf{1} - \mathbf{q}^m),$$

donde  $\circ$  denota el producto de Hadamard.

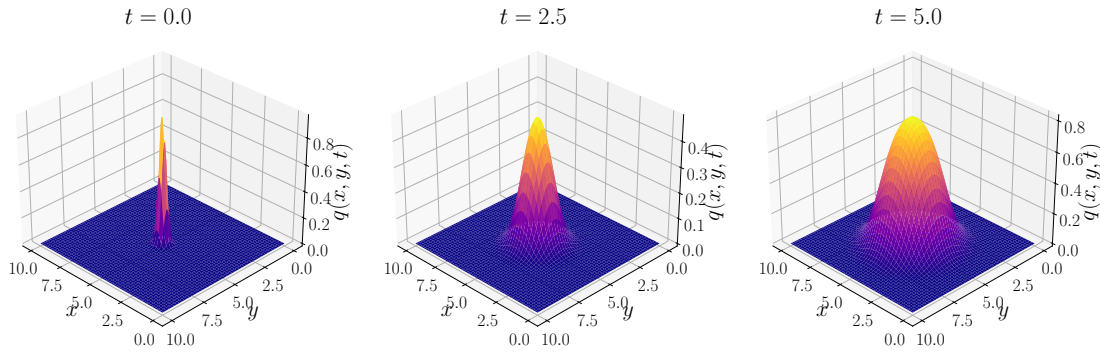
Al definir,

$$\alpha := \frac{\mathfrak{D} \Delta t}{2}, \quad \mathcal{A} := \mathbf{I} - \alpha \mathbf{L}, \quad \mathcal{B} := \mathbf{I} + \alpha \mathbf{L},$$

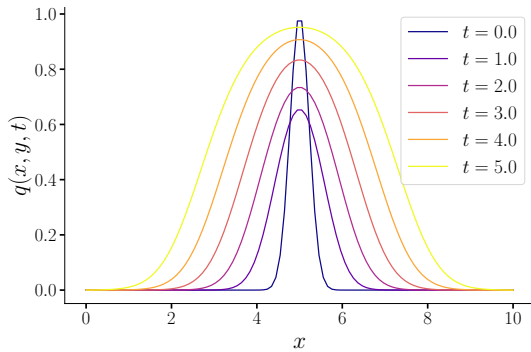
entonces en cada paso de tiempo

$\mathcal{A} \mathbf{q}^{m+1} = \mathcal{B} \mathbf{q}^m + \Delta t \rho \mathbf{q}^m \circ (\mathbf{1} - \mathbf{q}^m)$ . Al resolver este sistema lineal disperso obtenemos la solución del estado deseada en cada paso de tiempo, la cual utilizaremos para construir la matriz de instantáneas  $\mathbf{Q} \in \mathbb{R}^{(N_x N_y) \times N_t}$ .

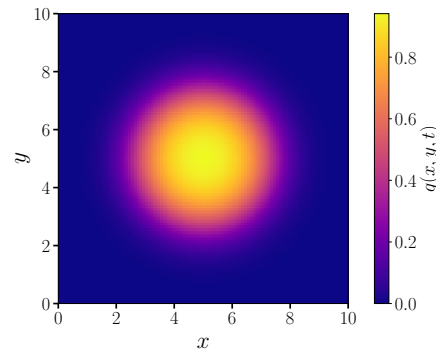
La visualización de la dinámica para la ecuación de Fisher-KPP se muestra en la Figura 4.8.



(a) Superficie 3D de  $q(x, y, t)$  para  $\mathfrak{D} = 0,1$  y  $\rho = 1$ .



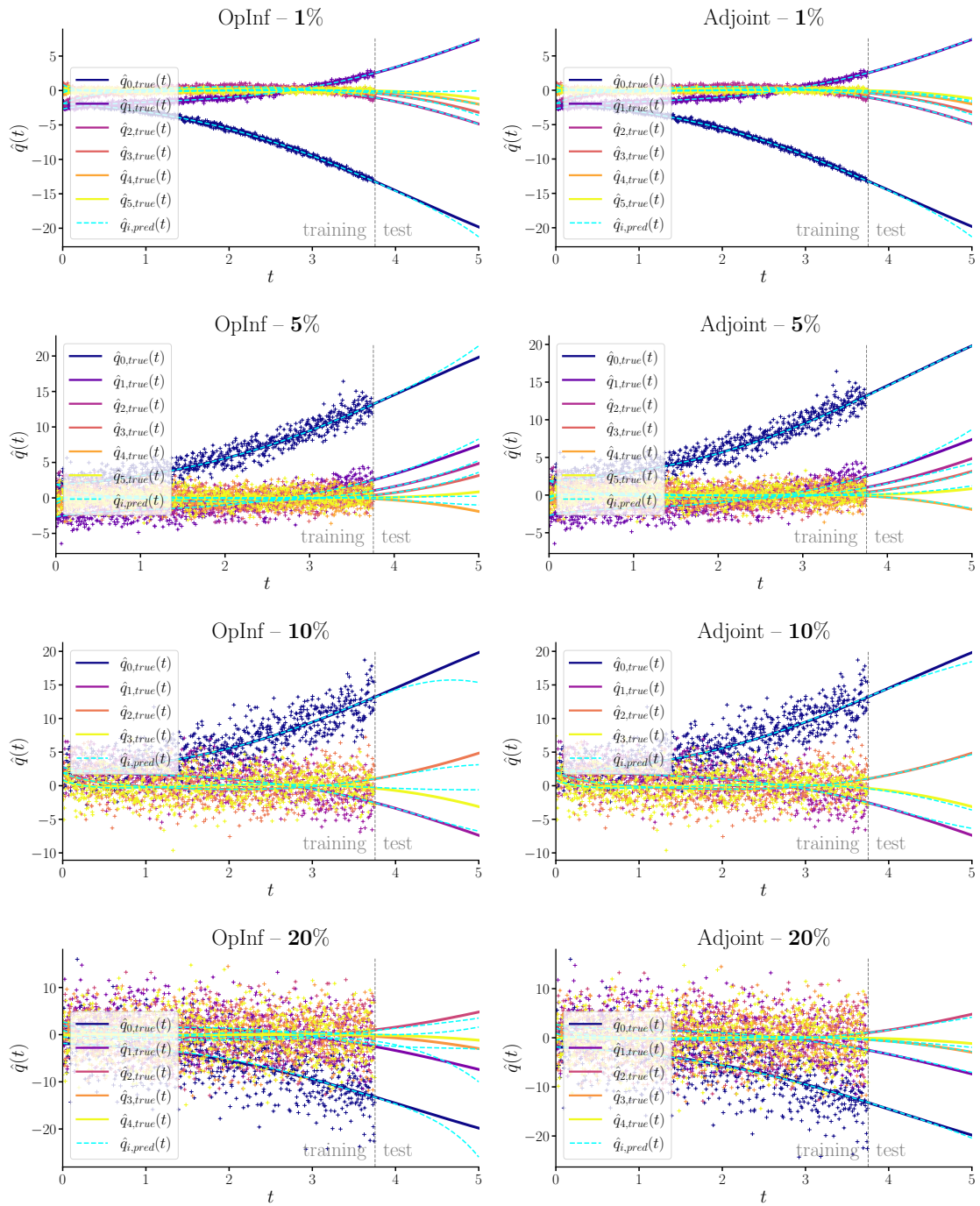
(b) Perfiles de  $q(x, y, t)$  para diferentes  $t_k$ .



(c) Contorno espacio-temporal de  $q(x, y, t)$  en  $t = 5$ .

**Figura 4.8:** Dinámica de la ecuación FKPP utilizada para la generación de datos.

### Perturbación por Ruido

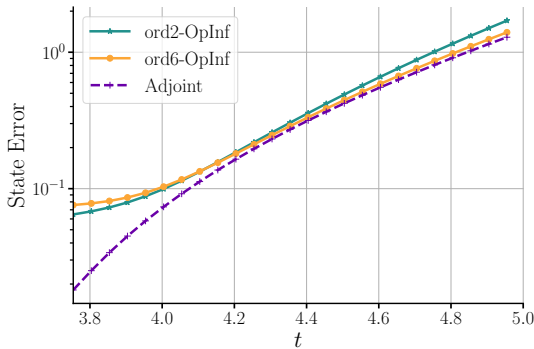


**Figura 4.9:** Simulaciones de perturbación por ruido para el experimento sintético de la ecuación de FKPP.

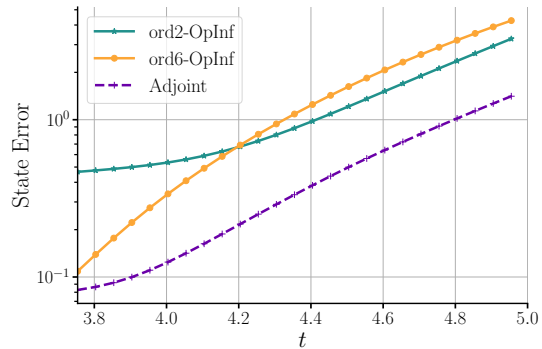
También llevamos a cabo el estudio de reducción de densidad de datos en la ecuación de Fisher-KPP 2D. Puesto que sus resultados reflejaron fielmente los de Burgers (mostrando que no existe una brecha de rendimiento significativa entre OpInf de sexto orden y el método adjunto), omitimos el análisis detallado aquí. En su lugar, nos centramos únicamente en los resultados de la perturbación por ruido, que se muestran en las Figuras 4.9, 4.10 y 4.11. Debido a consideraciones de estabilidad en la dinámica de reacción-difusión, en este caso ampliamos la ventana de entrenamiento para cubrir el 75 % del tiempo total de simulación.

A lo largo de los cuatro niveles de ruido (1 %, 5 %, 10 % y 20 %), el ROM entrenado mediante el método adjunto superó consistentemente a la mejor configuración de OpInf ('ord6'), tanto en la fidelidad de las trayectorias como en las métricas de error.

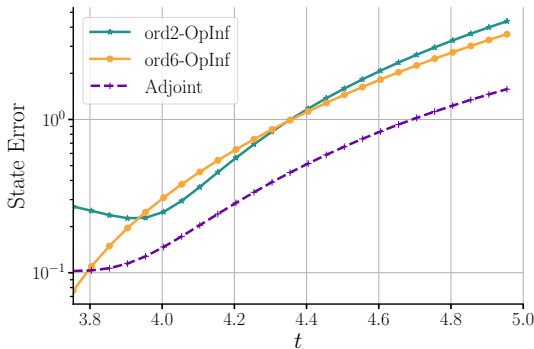
En las comparaciones de series temporales de los estados reducidos (Figura 4.10), las curvas de error de predicción correspondientes confirman que el enfoque adjunto mantiene una evolución del error más baja sobre el intervalo de predicción, especialmente en los niveles de ruido más altos, donde las estimaciones de la derivada por diferencias finitas se degradan significativamente.



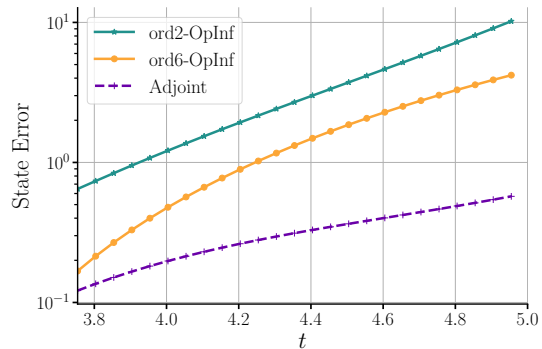
(a) 1 % de nivel de ruido.



(b) 5 % de nivel de ruido.



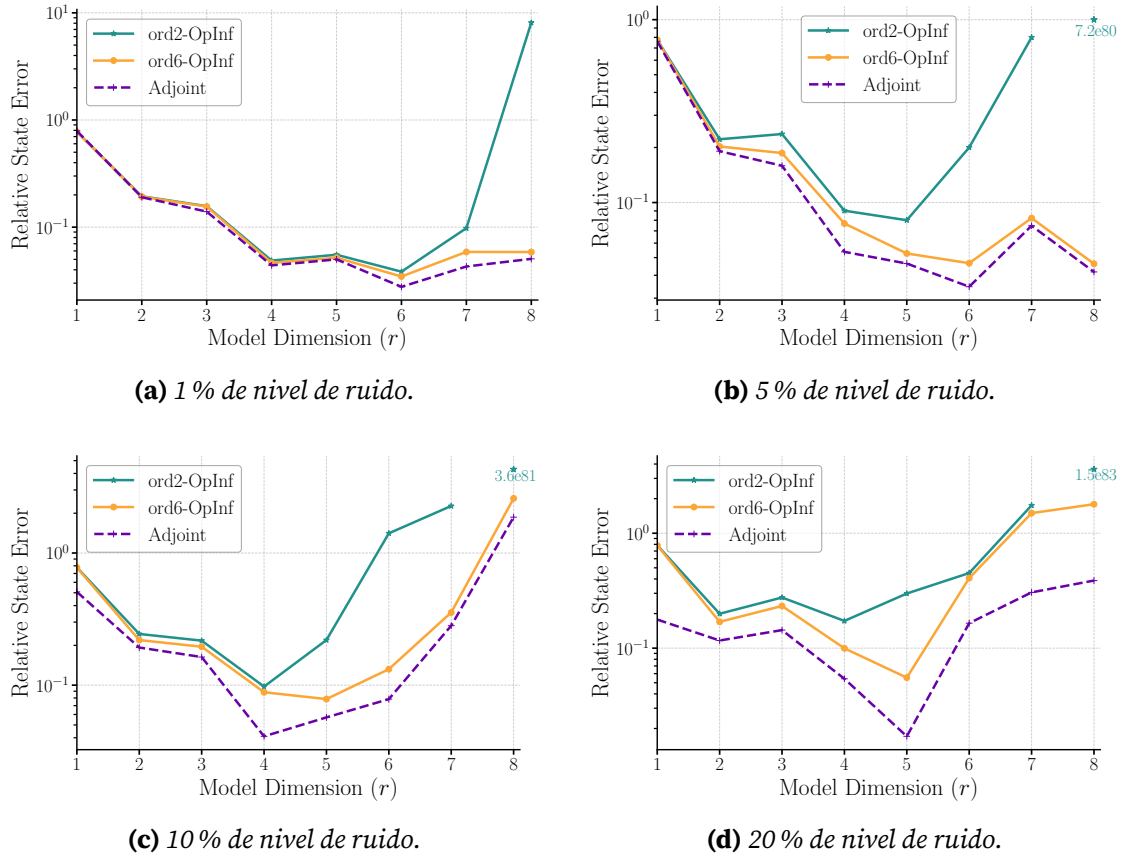
(c) 10 % de nivel de ruido.



(d) 20 % de nivel de ruido.

**Figura 4.10:** Error de predicción frente al tiempo para cada ejecución de nivel de ruido en el experimento de la ecuación de FKPP.

Finalmente, los gráficos de error relativo frente a la dimensión del modelo en la Figura 4.11 demuestran que la precisión del modelo entrenado mediante el método adjunto crece con el nivel de ruido, subrayando que el cómputo del gradiente exacto a través de este método produce actualizaciones paramétricas más fiables cuando las instantáneas de entrenamiento están corrompidas. En general, estos hallazgos reflejan y refuerzan los resultados obtenidos para Burgers; el método adjunto proporciona una robustez superior al ruido de medición sin sacrificar la viabilidad computacional.



**Figura 4.11:** Error relativo frente a  $r$  para cada ejecución de perturbación por ruido en el experimento de la ecuación de FKPP.



## DISCUSIÓN

En esta tesis hemos investigado el uso del entrenamiento basado en el estado adjunto para modelos de orden reducido (ROM) y lo hemos comparado con el enfoque estándar de OpInf. Nuestros experimentos se centraron en dos aspectos principales: la influencia de la densidad de los datos de entrenamiento y la robustez frente a datos ruidosos o imperfectos.

Un hallazgo clave es que la reducción de la densidad de los datos de las instantáneas (*snapshots*) tuvo un efecto mínimo en el rendimiento comparativo de OpInf con diferencias finitas (FD) de alto orden y el método adjunto. Cuando se varió el número de instantáneas de entrenamiento, tanto OpInf como los modelos entrenados mediante el método adjunto exhibieron un comportamiento en gran medida similar. Ninguno de los métodos mostró una ventaja clara a medida que disminuían las instantáneas, y los errores de predicción se mantuvieron comparables en todas las densidades probadas. Esto sugiere que, para los sistemas considerados en esta tesis, una reducción moderada en los datos de entrenamiento no eliminó información crítica. Comienzan a observarse resultados perceptibles ante reducciones extremas, como la del 0.2 % (solo 20 puntos de datos). Esta observación implica que, en ausencia de ruido, tanto el método adjunto como el método OpInf de alto orden requieren un número mínimo similar de instantáneas para identificar con precisión la dinámica dominante, lo que permite un submuestreo seguro en conjuntos de datos grandes.

En contraste con la densidad de datos, los experimentos con datos ruidosos o imperfectos revelaron una diferencia notable entre ambos métodos. El entrenamiento basado en el estado adjunto mostró una clara ventaja en términos de robustez. En todos los niveles de ruido añadido, los OpInf-ROM entrenados mediante el método adjunto alcanzaron consistentemente errores de predicción y errores relativos menores que los modelos OpInf basados en diferencias finitas. A medida que aumentaba el ruido en los datos de entrenamiento, el rendimiento del método OpInf estándar se degradaba más que el del modelo adjunto. Esto significa que, bajo las mismas condiciones de ruido, el

método adjunto produjo predicciones más fiables. Una razón de esta robustez es que el método adjunto ajusta los parámetros del modelo minimizando directamente una función de pérdida que tiene en cuenta la trayectoria completa. Al retropropagar el error a través de la dinámica temporal, tiende a filtrar las fluctuaciones espurias que surgen del ruido en las instantáneas individuales. En efecto, la optimización alinea el modelo reducido para ajustarse al comportamiento global del sistema en lugar de adaptarse exactamente a cada punto de datos ruidoso. Por el contrario, el enfoque OpInf estándar resuelve un problema estático de mínimos cuadrados sobre los datos de las instantáneas. El ruido en los datos puede distorsionar esta regresión, provocando que los coeficientes inferidos se sesguen debido al ruido y conduzcan a peores predicciones. La implicación práctica es que el método adjunto es muy adecuado para situaciones donde los datos disponibles son imperfectos, como las mediciones de experimentos o simulaciones truncadas. En estos casos, su resiliencia significa que se puede confiar en que el modelo reducido generalizará mejor, mientras que el modelo OpInf podría requerir un cuidado adicional (por ejemplo, filtrado de ruido o una regularización más fuerte) para evitar el sobreajuste al ruido. Esta ventaja del entrenamiento adjunto es especialmente importante en aplicaciones del mundo real con datos ruidosos o inciertos, donde obtener instantáneas perfectamente limpias puede resultar difícil.

Desde el punto de vista del coste computacional, el método adjunto añade una sobrecarga mínima, ya que escala de forma constante con el número de parámetros  $\hat{\theta} \in \mathbb{R}^d$  (solo es necesario resolver una EDO hacia atrás independientemente de  $d$ ), a diferencia de los enfoques de sensibilidad primal, que escalan de manera lineal.

Finalmente, vale la pena destacar la dependencia de una buena estimación inicial (*initial guess*) para el proceso de optimización basado en descenso de gradiente (GD) utilizado en el entrenamiento adjunto, la cual a su vez depende de una solución óptima del método OpInf. El método OpInf estándar es sensible a la estabilidad y al condicionamiento de su problema de regresión subyacente. Tres factores clave surgieron como elementos que influyen significativamente en esta estabilidad:

- (I) En primer lugar, la *estructura polinómica de los datos* es crucial. OpInf aprende los coeficientes para un conjunto elegido de términos de base polinómica del estado reducido. Si se incluyen demasiados términos de alto grado, o si las características están mal escaladas, la matriz de regresión puede llegar a estar mal condicionada.
- (II) En segundo lugar, la *selección de la base POD  $\mathbf{V}_r$*  es un factor crítico. Si esta base no logra capturar los modos importantes del sistema completo, los operadores inferidos serán inexactos o incluso inidentificables. Por el contrario, elegir un valor de  $r$  demasiado grande en relación con los datos puede amplificar el ruido y hacer que la inferencia esté mal condicionada. Por lo tanto, se debe equilibrar la dimensión de la base: muy pocos modos omitirán dinámicas clave, mientras que demasiados pueden desestabilizar la regresión.
- (III) En tercer lugar, el *uso de la regularización* suele ser necesario para combatir el mal

condicionamiento. Sin regularización, los valores singulares pequeños en la regresión pueden hacer que la solución tenga coeficientes excesivamente grandes o ambivalentes que se sobreajusten al ruido. Al añadir un término de penalización por regularización (por ejemplo,  $L^2$ , Tikhonov, ...), se puede estabilizar la inversión y asegurar que los operadores aprendidos permanezcan acotados. En nuestros experimentos, la introducción de una cantidad moderada de regularización mejoró la robustez de OpInf, a costa de un pequeño sesgo. Este compromiso generalmente vale la pena para obtener un modelo estable.

En resumen, nuestros hallazgos indican que tanto OpInf como el método basado en el estado adjunto pueden aprender modelos de orden reducido efectivos, pero sus fortalezas difieren bajo diversas condiciones. Reducir simplemente la densidad de las instantáneas no distinguió significativamente a los dos métodos, ya que ambos funcionaron de manera comparable con cantidades de datos variadas. Sin embargo, el método adjunto demostró ser mucho más robusto cuando los datos de entrenamiento eran ruidosos o imperfectos. En conjunto, estas ideas pueden guiar la selección práctica de los métodos de entrenamiento de ROM. Cuando la calidad de los datos es incierta o se necesita la mayor precisión, la resiliencia del método adjunto lo convierte en un candidato sólido, mientras que cuando los datos son abundantes y limpios, el enfoque OpInf estándar puede ser suficiente.



## CONCLUSIONES

En este trabajo, hemos desarrollado y analizado un marco de entrenamiento basado en el estado adjunto para modelos de orden reducido, formulado como una versión integral del problema estándar de Inferencia de Operadores (OpInf). Al plantear el aprendizaje de parámetros como una optimización de ajuste de trayectorias (*trajectory matching*) con un cómputo exacto del gradiente, el método adjunto ofrece varias ventajas clave frente a la inferencia basada en diferencias finitas. Nuestras principales conclusiones son las siguientes:

- (I) **Robustez frente a datos imperfectos.** El método adjunto supera consistentemente a OpInf cuando las instantáneas (*snapshots*) de entrenamiento están corrompidas por ruido. Debido a que minimiza una pérdida global a lo largo de todo el intervalo de tiempo, filtra las fluctuaciones espurias y produce estimaciones paramétricas más fiables bajo mediciones ruidosas o incompletas.
- (II) **Eficiencia en costes.** El entrenamiento basado en el estado adjunto implica una resolución directa (*forward*) y otra hacia atrás (*backward*) de manera independiente al número de parámetros  $d$ . Para modelos muy grandes o de alta fidelidad, este coste constante por iteración es más favorable que el de los enfoques de sensibilidad primal, cuyo coste crece linealmente con el número de parámetros.
- (III) **Flexibilidad y extensibilidad.** El marco adjunto se integra a la perfección con una gran variedad de esquemas de integración temporal y funcionales de pérdida, y puede incorporar fácilmente restricciones físicas adicionales o términos de regularización. Esta modularidad lo convierte en una herramienta versátil para construir modelos reducidos conscientes de la física (*physics-aware*) en diversos sistemas dinámicos.

A partir de los fundamentos presentados en esta tesis, identificamos varias direcciones prometedoras para estudios futuros:

- **Diferenciación automática para el cómputo de gradientes.** Sustituir las ecuaciones adjuntas deducidas a mano por una implementación de diferenciación automática (*autodiff*) podría simplificar el desarrollo de nuevos modelos reducidos y reducir los errores de implementación.
- **Análisis de eficiencia en costes computacionales.** Realizar un análisis comparativo (*benchmark*) de los métodos adjunto y OpInf evaluando diferentes tamaños de modelo, número de parámetros y horizontes temporales.
- **Extensión a EDP y condiciones de contorno adicionales.** Aplicar el método adjunto a otros sistemas desafiantes, tales como las ecuaciones de Euler o Navier-Stokes, o a problemas con condiciones iniciales variables (por ejemplo, condiciones de contorno periódicas) y acoplamientos multifísicos, para evaluar más a fondo su generalidad y robustez.

## REFERENCIAS

- [1] Peter Benner et al. «A Survey of Projection-Based Model Reduction Methods for Parametric Dynamical Systems». En: *SIAM Review* 57.4 (2015), págs. 483-531. DOI: 10.1137/130932715.
- [2] Alfio Quarteroni et al. *Reduced Basis Methods for Partial Differential Equations: An Introduction*. Cham, Switzerland: Springer, 2016.
- [3] Benjamin Peherstorfer et al. «Data-driven Operator Inference for Nonintrusive Projection Based Model Reduction». En: *Computer Methods in Applied Mechanics and Engineering* 306 (2016), págs. 196-215. DOI: <https://doi.org/10.1016/j.cma.2016.03.025>.
- [4] Omar Ghattas et al. «Learning Physics - Based Models from Data: Perspectives from Inverse Problems and Model Reduction». En: *Acta Numerica* 30 (2021). DOI: <https://doi.org/10.1017/S0962492921000064>.
- [5] Ricky T. Q. Chen et al. «Neural Ordinary Differential Equations». En: *Advances in Neural Information Processing Systems* 31 (2018). DOI: <https://doi.org/10.48550/arXiv.1806.07366>.
- [6] Harbir Antil et al. «A Brief Introduction to PDE-Constrained Optimization». En: *Frontiers in PDE-Constrained Optimization*. Springer, 2018, págs. 3-40. DOI: [https://doi.org/10.1007/978-1-4939-8636-1\\_1](https://doi.org/10.1007/978-1-4939-8636-1_1).
- [7] Andrew M. Bradley. *PDE-Constrained Optimization and the Adjoint Method*. Accessed: 20-Jan-2025. 2024. URL: [https://cs.stanford.edu/~ambrad/adjoint\\_tutorial.pdf](https://cs.stanford.edu/~ambrad/adjoint_tutorial.pdf).
- [8] Gal Berkooz et al. «The Proper Orthogonal Decomposition in the Analysis of Turbulent Flows». En: *Annual review of fluid mechanics* 25.1 (1993), págs. 539-575. DOI: <https://doi.org/10.1146/annurev.fl.25.010193.002543>.
- [9] Willcox Research Group. *Operator Inference for Data-Driven Reduced-Order Modeling*. Accessed: 16-Feb-2025. 2025. URL: <https://willcox-research-group.github.io/rom-operator-inference-Python3/source/opinf/intro.html>.

- [10] Boris Kramer et al. «Learning Nonlinear Reduced Models from Data with Operator Inference». En: *Annual Review of Fluid Mechanics* 56.1 (2024), págs. 521-548. DOI: <https://doi.org/10.1146/annurev-fluid-121021-025220>.
- [11] Clarence W. Rowley et al. «Model Reduction for Flow Analysis and Control». En: *Annual Review of Fluid Mechanics* 49.1 (2017), págs. 387-417. DOI: <https://doi.org/10.1146/annurev-fluid-010816-060042>.
- [12] C. De Boor. *A Practical Guide to Splines*. New York, NY: Springer-Verlag, 1978.
- [13] J. Frédéric Bonnans et al. *Perturbation Analysis of Optimization Problems*. New York, NY: Springer Science & Business Media, 2013.
- [14] Paolo Luchini et al. «An Introduction to Adjoint Problems». En: *arXiv preprint arXiv:2404.17304* (2024). DOI: <https://doi.org/10.48550/arXiv.2404.17304>.
- [15] Shane A. McQuarrie et al. «Bayesian Learning with Gaussian Processes for Low-Dimensional Representations of Time-Dependent Nonlinear Systems». En: *Physica D: Nonlinear Phenomena* 475 (2025), pág. 134572. ISSN: 0167-2789. DOI: <https://doi.org/10.1016/j.physd.2025.134572>.
- [16] Jorge Nocedal et al. *Numerical Optimization*. New York, NY: Springer, 1999.
- [17] Daniel Liberzon. *Calculus of Variations and Optimal Control Theory: A Concise Introduction*. Princeton, NJ: Princeton University Press, 2011.
- [18] Sebastian Ruder. *An Overview of Gradient Descent Optimization Algorithms*. Accessed: 16-Feb-2025. 2017. URL: <https://arxiv.org/abs/1609.04747>.
- [19] B. Sengupta et al. «Efficient Gradient Computation for Dynamical Models». En: *NeuroImage* 98 (2014), págs. 521-527. ISSN: 1053-8119. DOI: <https://doi.org/10.1016/j.neuroimage.2014.04.040>.
- [20] Guillaume Garrigos et al. «Handbook of Convergence Theorems for (Stochastic) Gradient Methods». En: *arXiv preprint arXiv:2301.11235* (2023). DOI: <https://doi.org/10.48550/arXiv.2301.11235>.
- [21] Pauli Virtanen et al. «SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python». En: *Nature Methods* 17.3 (2020), págs. 261-272. DOI: [10.1038/s41592-019-0686-2](https://doi.org/10.1038/s41592-019-0686-2).
- [22] SciPy Developers. ‘*scipy.integrate.solve\_ivp*’ — *SciPy v1.x Manual*. [https://docs.scipy.org/doc/scipy/reference/generated/scipy.integrate.solve\\_ivp.html](https://docs.scipy.org/doc/scipy/reference/generated/scipy.integrate.solve_ivp.html). Accessed: 2025-03-09. 2025.
- [23] Tamara G. Kolda et al. «Tensor Decompositions and Applications». En: *SIAM Review* 51.3 (2009), págs. 455-500. DOI: <https://doi.org/10.1137/07070111X>.

# Apéndices



# FUNDAMENTOS TEÓRICOS ADICIONALES

## A.1. Matricización de Tensores

En los modelos de orden reducido, el operador no lineal

$$\mathcal{H} : \mathbb{R}^r \times \mathbb{R}^r \rightarrow \mathbb{R}^r$$

que genera la contribución cuadrática

$$[\mathcal{H}(\hat{\mathbf{q}}(t), \hat{\mathbf{q}}(t))]_i = \sum_{j=1}^r \sum_{k=1}^r H_{i,j,k} \hat{q}_j(t) \hat{q}_k(t),$$

puede interpretarse como un tensor de tercer orden  $\mathcal{H} \in \mathbb{R}^{r \times r \times r}$ . Para evaluar este término de manera eficiente, *matricizamos* el tensor en una matriz que actúa sobre el producto de Kronecker  $\hat{\mathbf{q}}(t) \otimes \hat{\mathbf{q}}(t) \in \mathbb{R}^{r^2}$  [23].

El *despliegue en el modo 1* (matricización)  $H_{(1)} \in \mathbb{R}^{r \times r^2}$  de  $\mathcal{H}$  se define de tal manera que

$$H_{(1)} (\hat{\mathbf{q}}(t) \otimes \hat{\mathbf{q}}(t)) = \left[ \sum_{j,k} H_{1,j,k} \hat{q}_j \hat{q}_k, \dots, \sum_{j,k} H_{r,j,k} \hat{q}_j \hat{q}_k \right]^\top = \mathcal{H}(\hat{\mathbf{q}}(t), \hat{\mathbf{q}}(t)).$$

Concretamente, si aplanamos  $\hat{\mathbf{q}}(t)$  en un vector de longitud  $r^2$  apilando sus componentes en orden mayor por columnas (\*column-major order\*), entonces

$$[H_{(1)}]_{i, (j-1)r+k} = H_{i,j,k},$$

y

$$\mathcal{H}(\hat{\mathbf{q}}, \hat{\mathbf{q}}) = H_{(1)} \text{vec}(\hat{\mathbf{q}} \hat{\mathbf{q}}^\top) = \hat{\mathbf{H}}(\hat{\mathbf{q}} \otimes \hat{\mathbf{q}}).$$

### Ejemplo: Ecuación de Burgers

Considere la ecuación de Burgers viscosa 1D para el campo escalar  $q(x, t) : [0, 1] \times [0, T] \rightarrow \mathbb{R}$ :

$$\frac{\partial q}{\partial t} + q \frac{\partial q}{\partial x} = \nu \frac{\partial^2 q}{\partial x^2}.$$

Tras la discretización espacial en  $n$  puntos de la malla, escribimos

$$\mathbf{q}(t) = [q(x_1, t), q(x_2, t), \dots, q(x_n, t)]^T \in \mathbb{R}^n.$$

Por lo tanto, el término convectivo de Burgers del modelo completo (\*FOM\*) resulta en

$$\mathbf{N}(\mathbf{q}) = -\mathbf{q} \circ (\mathbf{D}_x \mathbf{q}) \in \mathbb{R}^n,$$

donde  $\circ$  denota el producto de Hadamard (componente a componente) y  $\mathbf{D}_x \in \mathbb{R}^{n \times n}$  es la matriz de diferencias finitas para el operador  $\partial_x$ . Tras la proyección POD, planteamos el \*ansatz\* reducido como

$$\mathbf{q}(t) \approx \Phi \hat{\mathbf{q}}(t), \quad \Phi \in \mathbb{R}^{n \times r}, \quad \hat{\mathbf{q}} \in \mathbb{R}^r.$$

Sustituyendo en  $\mathbf{N}$  y proyectando sobre la base obtenemos

$$\Phi^T \mathbf{N}(\Phi \hat{\mathbf{q}}) = -\Phi^T [(\Phi \hat{\mathbf{q}}) \circ (\mathbf{D}_x \Phi \hat{\mathbf{q}})].$$

Esto puede expresarse como una forma bilineal utilizando la identidad  $(a \circ b)_\ell = \sum_{j,k} \delta_{\ell j} a_j \delta_{\ell k} b_k$ .

A partir de ella, se puede demostrar que

$$[\Phi^T (\Phi \hat{\mathbf{q}} \circ \mathbf{D}_x \Phi \hat{\mathbf{q}})]_i = \sum_{j=1}^r \sum_{k=1}^r \underbrace{\sum_{\ell=1}^n \Phi_{\ell,i} \Phi_{\ell,j} [\mathbf{D}_x \Phi]_{\ell,k}}_{\mathcal{H}_{i,j,k}} \hat{q}_j \hat{q}_k.$$

Por lo tanto, los coeficientes

$$\mathcal{H}_{i,j,k} = - \sum_{\ell=1}^n \Phi_{\ell,i} \Phi_{\ell,j} [\mathbf{D}_x \Phi]_{\ell,k}, \quad i, j, k = 1, \dots, r,$$

definen un tensor de tercer orden  $\mathcal{H} \in \mathbb{R}^{r \times r \times r}$ . El despliegue en el modo 1 (matricización) proporciona la matriz correspondiente  $H_{(1)} \in \mathbb{R}^{r \times r^2}$  con componentes

$$[H_{(1)}]_{i, (j-1)r+k} = \mathcal{H}_{i,j,k}, \quad 1 \leq i, j, k \leq r,$$

de modo que

$$\Phi^T \mathbf{N}(\Phi \hat{\mathbf{q}}) = H_{(1)} (\hat{\mathbf{q}} \otimes \hat{\mathbf{q}}) = \hat{\mathbf{H}} (\hat{\mathbf{q}} \otimes \hat{\mathbf{q}}).$$

Aquí,  $\hat{\mathbf{H}} \in \mathbb{R}^{r \times r^2}$  codifica las interacciones no lineales proyectadas. Esta forma matricizada permite un cómputo eficiente de  $\hat{\mathbf{H}}$  durante la inferencia de operadores, preservando al mismo tiempo la estructura no lineal cúbica.

## A.2. Interpolación por Splines Cúbicos

Los splines cúbicos son funciones polinómicas a trozos que garantizan la suavidad y la continuidad hasta la segunda derivada en los puntos de datos. Dados  $N$  puntos  $\{(t_i, u_i)\}_{i=1}^N$ , el spline cúbico  $S(t)$  se define a trozos como:

$$S(t) = S_i(t), \quad t \in [t_i, t_{i+1}]$$

donde cada  $S_i(t)$  es un polinomio cúbico:

$$S_i(t) = a_i + b_i(t - t_i) + c_i(t - t_i)^2 + d_i(t - t_i)^3.$$

Los coeficientes  $a_i, b_i, c_i, d_i$  se determinan mediante las siguientes condiciones:

- Condición de interpolación:  $S_i(t_i) = u_i$  y  $S_i(t_{i+1}) = u_{i+1}$ ,
- Continuidad de la primera y segunda derivada en los nodos internos:

$$S'_i(t_{i+1}) = S'_{i+1}(t_{i+1}), \quad S''_i(t_{i+1}) = S''_{i+1}(t_{i+1}),$$

- Condiciones de contorno: bien spline *natural* ( $S''_1(t_1) = 0$  y  $S''_{N-1}(t_N) = 0$ ) o bien spline *sujeto* o *clamped* (primeras derivadas prescritas en los extremos del dominio).

Los splines cúbicos aseguran la suavidad y proporcionan una interpolación natural que evita las oscilaciones típicas de la interpolación polinómica de alto grado [12]. Las condiciones de suavidad y continuidad impuestas por los splines cúbicos —específicamente, la existencia de primera y segunda derivada continuas— son esenciales para la aplicación del método adjunto (Sección 3.2). El método adjunto requiere que la trayectoria interpolada  $\hat{\mathbf{q}}(t)$  sea al menos de clase  $\mathcal{C}^1$  para asegurar el buen planteamiento de las ecuaciones adjuntas y un cálculo preciso del gradiente. Los splines cúbicos satisfacen este requisito, ya que su continuidad  $\mathcal{C}^2$  garantiza que  $\hat{\mathbf{q}}(t)$  es dos veces diferenciable en todo el dominio  $\mathcal{T}$ . Esto evita inestabilidades numéricas que podrían surgir debido a derivadas discontinuas al resolver el sistema adjunto hacia atrás en el tiempo. Al imponer la continuidad de las derivadas en los nodos, los splines cúbicos preservan la estructura necesaria para aplicar la regla de la cadena en (3.6), permitiendo una optimización eficiente basada en gradiente del funcional de pérdida en tiempo continuo (3.1).



## CÓDIGO EN PYTHON

## Ecuación de Burgers

**Listing B.1:** Resolución de la ecuación de Burgers en 1D. Generación de instantáneas de datos (Q)

```

1 import numpy as np
2 from scipy.sparse import diags
3 from scipy.sparse.linalg import spsolve
4
5 # Grid and time parameters
6 M = 9999 # Number of time steps
7 T = 1.0 # Final time
8 dt = T / M # Time step size
9
10 #N = 998 # Number of spatial grid points
11 N = 2**7 # Number of spatial grid points
12 L = 1.0 # Domain length
13 dx = L / (N + 1) # Spatial step size (excluding boundary points)
14
15 x = np.linspace(0, L, N+2) # Includes boundary points
16 t = np.linspace(0, T, M+1)
17
18 # Diffusivity (viscosity)
19 nu = 0.01
20
21 # Initial condition  $q(x,0) = \sin(x)$  (excluding boundaries)
22 q0 = np.sin(2*np.pi * x[1:-1])
23
24 # Solution matrix initialization (excluding boundaries)
25 Q = np.zeros((N, M+1))
26 Q[:, 0] = q0
27
28 # Finite Difference Matrix (Diffusion Term)
29 off_diag = np.full(N-1, 1 / dx**2)
30 main_diag = np.full(N, -2 / dx**2)

```

```

31
32 Tdx = diags([off_diag, main_diag, off_diag], [-1, 0, 1], shape=(N, N)).
    tocsr()
33
34 # Function to compute next time step using implicit diffusion
35 def viscous_burgers(qold, nu, dt, dx):
36     n = len(qold)
37     dx2 = dx**2
38
39     # Lax-Wendroff for nonlinear convection term
40     qjpone = np.roll(qold, -1)
41     qjmone = np.roll(qold, 1)
42
43     LW = qold - (dt / (2 * dx)) * qold * (qjpone - qjmone) + \
44         (dt**2 / (2 * dx2)) * qold * (0.5 * (qjpone - qjmone)**2 + \
45         qold * (qjpone - 2 * qold + qjmone))
46
47     # Apply Dirichlet BC: q(0,t) = q(1,t) = 0
48     LW[0] = 0
49     LW[-1] = 0
50
51     # Implicit diffusion term
52     A = np.eye(n) - nu * (dt / 2) * Tdx.toarray()
53     rhs = LW + nu * (dt / 2) * (Tdx @ qold)
54
55     # Solve linear system
56     qnew = spsolve(A, rhs)
57
58     # Enforce boundary conditions
59     qnew[0] = 0
60     qnew[-1] = 0
61
62     return qnew
63
64 # Time stepping
65 qold = q0.copy()
66 for i in range(M):
67     qnew = viscous_burgers(qold, nu, dt, dx)
68     Q[:, i+1] = qnew
69     qold = qnew
70
71 # Extend solution to include boundary points (q=0 at x=0 and x=1)
72 Qfom = np.zeros((N+2, M+1))
73 Qfom[1:-1, :] = Q # Fill interior points

```

## Ecuación de Fisher-KPP

**Listing B.2:** Resolución de la ecuación de Fisher-KPP bidimensional. Generación de (Q)

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.sparse import diags, kron, eye, csc_matrix
4 from scipy.sparse.linalg import spsolve
5
6 # Parameters
7 Lx = 10.0          # Domain length in x-direction
8 Ly = 10.0          # Domain length in y-direction
9 Nx = 100           # Number of spatial points in x-direction
10 Ny = 100           # Number of spatial points in y-direction
11 dx = Lx / (Nx - 1) # Spatial step in x
12 dy = Ly / (Ny - 1) # Spatial step in y
13 D = 0.1            # Diffusion coefficient
14 r = 1.0            # Growth rate
15 dt = 0.005         # Time step
16 T = 5.0            # Total simulation time
17 Nt = int(T / dt)   # Number of time steps
18
19 # Spatial grids
20 x = np.linspace(0, Lx, Nx)
21 y = np.linspace(0, Ly, Ny)
22 X, Y = np.meshgrid(x, y)
23
24 # Initial condition: 2D Gaussian at the center
25 u0 = np.exp(-10 * ((X - Lx/2)**2 + (Y - Ly/2)**2))
26 u = u0.flatten()   # Flatten to 1D vector
27
28 # Function to construct 1D Laplacian matrix with Neumann BCs
29 def construct_1d_laplacian(N, d):
30     main_diag = -2 * np.ones(N) / d**2
31     super_diag = np.ones(N-1) / d**2
32     sub_diag = np.ones(N-1) / d**2
33     # Neumann BC adjustments
34     super_diag[0] = 2 / d**2
35     sub_diag[-1] = 2 / d**2
36     return diags([sub_diag, main_diag, super_diag], [-1, 0, 1], format="csc")
37
38 # Construct 1D Laplacians
39 Lx = construct_1d_laplacian(Nx, dx)
40 Ly = construct_1d_laplacian(Ny, dy)
41
42 # Construct 2D Laplacian using Kronecker products
43 Ix = eye(Nx, format="csc")
44 Iy = eye(Ny, format="csc")
45 L = kron(Iy, Lx) + kron(Ly, Ix)
46

```

```

47 # Crank-Nicolson matrix
48 alpha = D * dt / 2
49 A = eye(Nx * Ny) - alpha * L
50 A = A.tocsc()
51
52 # Store solutions at certain time steps
53 snapshot_interval = 1 # Desired number of interval snapshot to store
54 snapshots = []
55
56 # Time stepping
57 for tt in range(1, Nt+1):
58     u_prev = u.copy()
59     reaction = r * u_prev * (1 - u_prev)
60     b = u_prev + dt * reaction + alpha * L.dot(u_prev)
61     u = spsolve(A, b)
62
63     if tt % snapshot_interval == 0:
64         snapshots.append(u.reshape(Ny, Nx))
65
66 # Convert snapshots to numpy array
67 Qfom = np.array(snapshots)

```

## Método del Estado Adjunto

**Listing B.3:** Algoritmo del Método Adjunto.

```

1 import numpy as np
2 import opinf
3 from scipy.integrate import cumulative_trapezoid
4
5 ##### OpInf #####
6 #####
7
8 opinf.utils.mpl_config()
9
10 # Parameters
11 r = 10
12 k_samples = 1000 # number of samples (snapshot data)
13 T = 1.0
14
15 # Snapshot data Q = [q(t_0) q(t_1) ... q(t_k)], size=(r,k)
16 Q = np.load("burgers_snapshots.npy")
17 t = np.load("burgers_time.npy") # t_0 = 0, ..., t_k = 1
18 x = np.load("burgers_space.npy")
19
20 dt = t[1] - t[0]
21
22 # Initialize orthonormal basis V
23 Vr = opinf.basis.PODBasis(cumulative_energy=0.999)

```

```

24 #Vr = opinf.basis.PODBasis(num_vectors=r)
25 Vr.fit(Q)
26 print(Vr)
27
28 # Compressed state snapshots Q_ by projecting onto the reduced space
   defined by Vr
29 Q_ = Vr.compress(Q)
30
31 # Estimate time derivatives using 6-thorder finite differences.
32 ddt_estimator = opinf.ddt.UniformFiniteDifferencer(t, "ord6")
33 Qdot_ = ddt_estimator.estimate(Q_)[1]
34
35 # Build the quadratic continuous model and fit it.
36 model = opinf.models.ContinuousModel(operators=["A", "H"])
37 model.fit(states=Q_, ddts=Qdot_)
38 A_opinf = model.operators[0].entries
39 H_opinf = model.operators[1].entries
40
41 # Expanded H: size (r, r^2)
42 H_opinf = opinf.operators.QuadraticOperator.expand_entries(H_opinf)
43
44 # Flatten A and H to build initial guess for theta.
45 # Start from the OpInf solution.
46 theta = np.concatenate([A_opinf.ravel(), H_opinf.ravel()])
47
48 ##### RHS #####
49 #####
50
51 # Integral of q(tau) from 0 to t_k for each t_k.
52 Q_int = np.zeros_like(Q_)
53 for i in range(r):
54     Q_int[i, :] = cumulative_trapezoid(Q_[i, :], t, initial=0)
55
56 # Quantity q(tau) q(tau) for each tau.
57 Q2_ = np.zeros((r**2, k_samples))
58 for k in range(k_samples):
59     q_k = Q[:, k]
60     Q2_[:, k] = np.kron(q_k, q_k)
61
62 # Integral of q(tau) q(tau) from 0 to t_k.
63 Q2_int = np.zeros_like(Q2_)
64 for i in range(r**2):
65     Q2_int[i, :] = cumulative_trapezoid(Q2_[i, :], t, initial=0)
66
67 ##### GD Parameters #####
68 #####
69
70 max_iter = 1000
71 epsilon = 1e-8 # stopping threshold for gradient norm
72
73 # Armijo parameters:

```

```

74 eta = 1e-3          # initial learning rate
75 alpha = 1e-4
76 beta = 0.5
77 d = r**2 + r**3
78
79 ##### GD Loop #####
80 #####
81
82 for j in range(max_iter):
83     A = theta[:r**2].reshape(r, r)
84     H = theta[r**2:].reshape(r, r**2)
85     q0 = Q[:, 0]
86
87     # Compute predicted states:  $\tilde{Q} = q_0 + A @ Q_{int} + H @ Q2_{int}$ .
88     tilde_Q = q0[:, np.newaxis] + A @ Q_int + H @ Q2_int
89
90     # Loss computation: mean squared error.
91     loss = np.mean(np.sum((Q_ - tilde_Q)**2, axis=0))
92     print(f"Iteration {j}, Loss: {loss:.6f}")
93
94     # Solve adjoint ODE backwards using implicit Euler.
95     lambda_values = np.zeros((r, k_samples))
96     lambda_values[:, -1] = 0.0 # Terminal condition:  $(T)=0$ 
97
98     for k in reversed(range(k_samples - 1)):
99         q_k = Q[:, k]
100         tilde_q_k = tilde_Q[:, k]
101         error_k = q_k - tilde_q_k
102
103         # Compute  $M(q_k)$ 
104         H_3d = H.reshape(r, r, r)
105         M_k = np.einsum("ijk,j->ki", H_3d, q_k)
106
107         # Backward Euler step.
108         matrix = np.eye(r) + dt * (A.T + 2 * M_k)
109         rhs = lambda_values[:, k+1] - 2 * dt * error_k
110         lambda_values[:, k] = np.linalg.solve(matrix, rhs)
111
112     # Gradient computation.
113     grad_A = np.zeros(r**2)
114     grad_H = np.zeros(r**3)
115
116     for k in range(k_samples):
117         lambda_k = lambda_values[:, k]
118         q_k = Q[:, k]
119
120         # Gradient parts for A.
121         outer_A = np.outer(lambda_k, q_k).flatten()
122         grad_A += outer_A * dt
123
124         # Gradient parts for H.

```

```

125     q_outer = np.outer(q_k, q_k).flatten()
126     outer_H = np.outer(lambda_k, q_outer).flatten()
127     grad_H += outer_H * dt
128
129     gradient = np.concatenate([grad_A, grad_H])
130     grad_norm = np.linalg.norm(gradient)
131
132     if grad_norm < epsilon:
133         print("Gradient norm below tolerance; stopping descent.")
134         break
135
136     # Armijo backtracking line search
137     eta_current = eta * 1.05 # Initial trial step size
138     ls_success = False
139     for _ in range(50): # Max line search iterations
140         theta_new = theta - eta_current * gradient
141         A = theta_new[:r**2].reshape(r, r)
142         H = theta_new[r**2:].reshape(r, r**2)
143         tilde_Q = q0[:, np.newaxis] + A @ Q_int + H @ Q2_int
144         loss_new = np.mean(np.sum((Q_ - tilde_Q)**2, axis=0))
145
146         if loss_new <= loss - alpha * eta_current * (grad_norm ** 2):
147             eta = eta_current
148             theta = theta_new
149             ls_success = True
150             break
151         else:
152             eta_current *= beta
153
154     if not ls_success:
155         theta = theta - eta * gradient # Fallback to previous eta
156         A = theta[:r**2].reshape(r, r)
157         H = theta[r**2:].reshape(r, r**2)
158         tilde_Q = q0[:, np.newaxis] + A @ Q_int + H @ Q2_int
159         loss_new = np.mean(np.sum((Q_ - tilde_Q)**2, axis=0))
160         if loss_new > loss:
161             eta *= beta # Force reduce eta if line search failed
162
163
164     # Detach optimal A and H from theta.
165     A_opt = theta[:r**2].reshape(r, r)
166     H_opt = theta[r**2:].reshape(r, r**2)

```





Trabajo de Fin de Máster en Ciencias Matemáticas 2025:E20  
ISSN 1404-6342

LUNFNA-3045-2025

Análisis Numérico  
Centro de Ciencias Matemáticas  
Universidad de Lund  
Box 118, SE-221 00 Lund, Suecia

<http://www.maths.lu.se/>